

LLM Application을 위한 AI Agent 설계 입문

LGCNS Gen.AI 교육

강사 김형욱
(hukim@artiasolution.com)

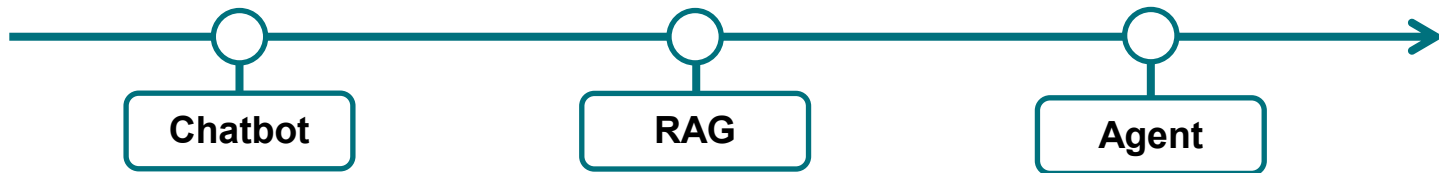
Contents

- AI 에이전트 개요 (3 page)
- 에이전트 동작 메커니즘 (11 page)
- 에이전트 설계 (44page)
- 부록. Agentic patterns (69 page)

AI 에이전트 개요

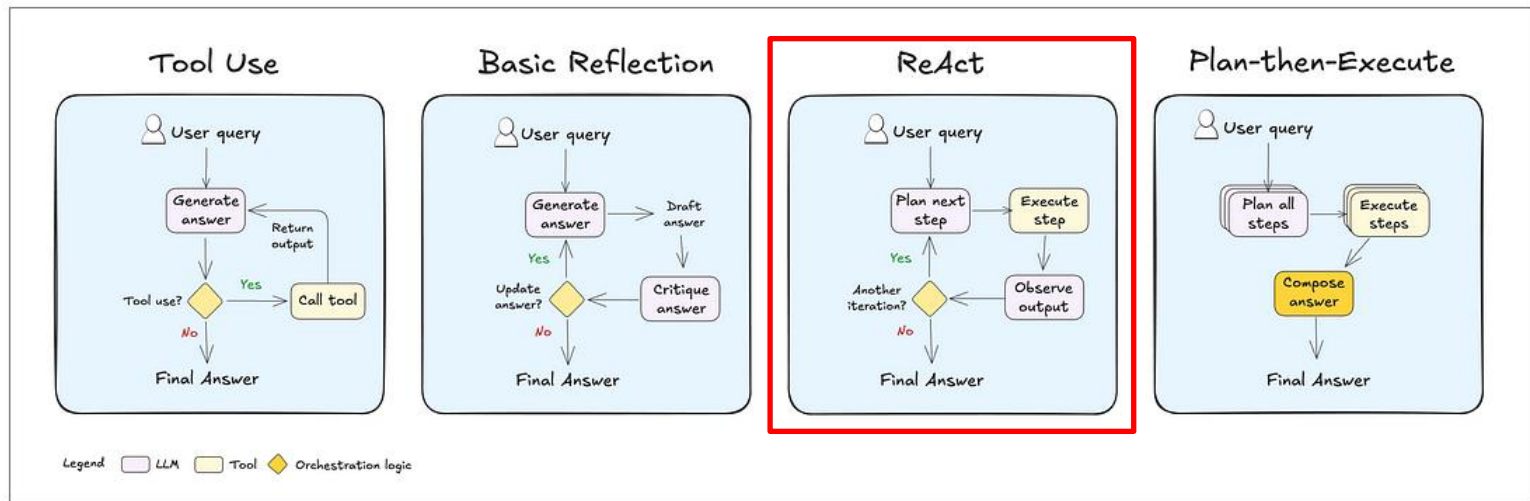
LLM에서 에이전트로 진화

- LLM Application의 진화: 컨텍스트의 확장
 - 1단계: Chatbot :
 - ✓ **대화 컨텍스트**로 이전 대화의 흐름을 기억하고 문맥에 맞는 답변을 생성
 - 2단계: RAG (Retrieval Augmented Generation)
 - ✓ **검색 컨텍스트**로 정보의 정확성을 높인 지식 확장형 질의응답
 - 3단계: 에이전트 (Agent)
 - ✓ **도구 컨텍스트**로 외부 시스템과 상호작용하며 자율적 과업 수행



AI 에이전트

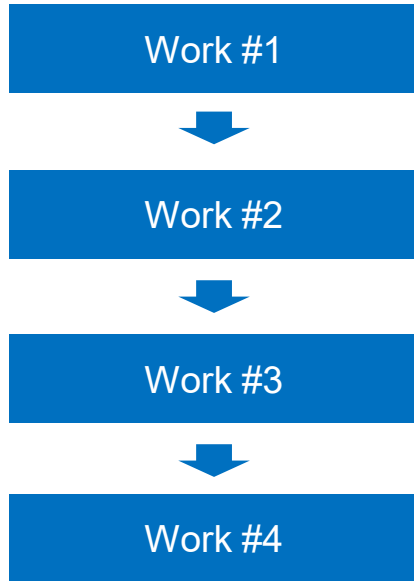
- 단순한 자동화를 넘어, LLM의 인지와 추론을 통해 상황을 판단하고 목표 달성을 위한 행동을 스스로 결정하고 수행하는 시스템
- 목표와 용도에 따라 다양한 Agentic Pattern 적용 가능
 - Tool Use, Reflection, **ReAct**, Plan-then-Execute, Multi-Agent, Agentic RAG / Agentic Workflow 등



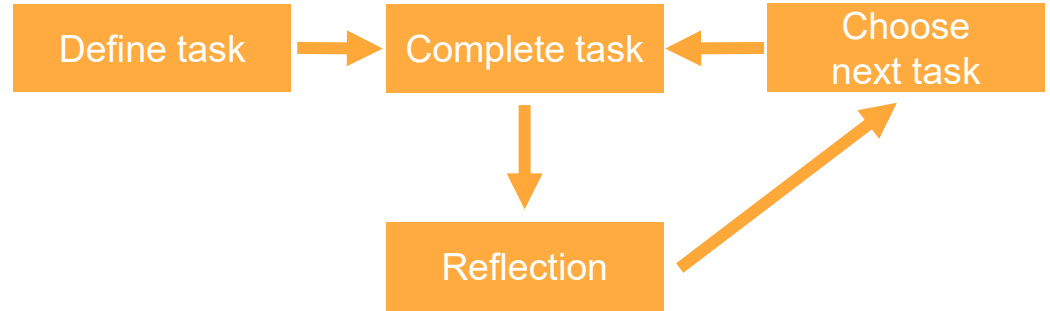
비교 : 워크플로우 vs 에이전트

- 워크플로우와 에이전트는 하나의 '에이전틱 시스템'을 이루는 상호 보완적인 두 요소
 - 워크플로우 (미리 짜인 '레시피')
 - ✓ 정해진 규칙에 따라 프로세스를 수행하는 예측 가능한 자동화 AI
 - 에이전트 (상황에 맞게 작업하는 '셰프')
 - ✓ 스스로 계획을 설정하여 과업을 동적으로 수행하는 자율적 AI

Workflow



AI Agent



구분	워크플로우 (Workflow)	AI 에이전트 (AI Agent)
적용 대상	잘 정의된 반복적이고 예측 가능한 정형화된 업무	업무 영역은 있지만 판단에 따른 유연한 작업 수행이 필요한 업무
결정 방식	Deterministic outcomes 명시된 조건(if-else)에 따라 동작	problem solving 실시간 판단 기반
유연성	낮음 (정해진 시나리오에 최적화)	높음 (새로운 상황에 유연하게 대처 가능)
장점	높은 제어력 신뢰성 비용 효율성	높은 유연성/적응성 복잡한 문제 해결 능력 학습 및 개인화
단점	명확한 요구조건 필요 지정된 작업만 가능	높은 비용 결과의 불투명성

에이전트 AI

- AI 에이전트 더 높은 수준의 자율성 / 문제 해결 능력 / 적응력 요구
 - AI 스스로 전체 워크플로우를 완전 자동화할 수 있는 시스템
- 목표 수행을 위해 스스로 계획·학습하며, 동시에 윤리적이고 고급 맥락 판단 능력 필요
 - 에이전트 관련 고급 기법 동원
 - ✓ 고급 추론 모델
 - ✓ 멀티 에이전트 오케스트레이션
 - ✓ 특정 업무에 전문적인 AI 에이전트
 - ✓ 컨텍스트 엔지니어링
 - ✓ 가드레일 & 모니터링 기술

에이전트 행동(Agentic Behaviors)

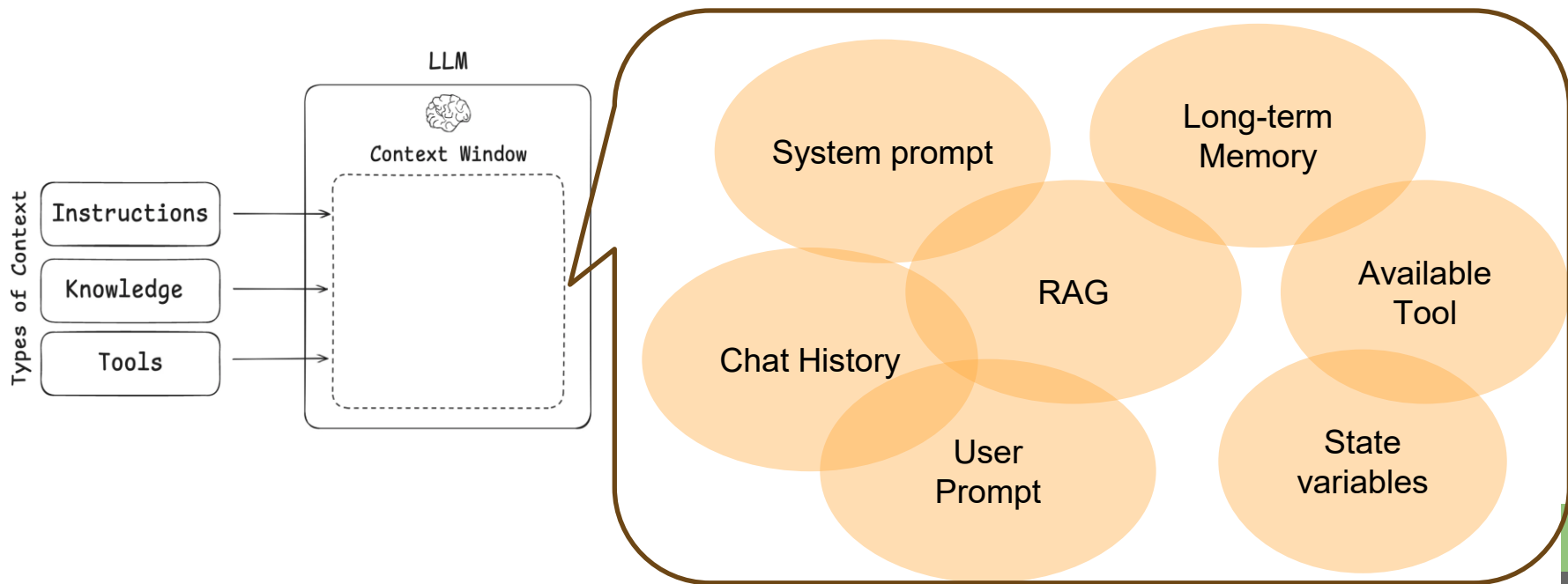
- 에이전트의 여부(O/X)보다 얼마나 '**Agentic**'한 시스템인가를 논의해야 함.
 - 목표 지향성(Goal-driven): 특정 목표를 달성하기 위한 논리적 사고와 계획 수립
 - 자율성(Autonomy): 인간의 개입 없이 독립적으로 의사결정
 - 적응성(Adaptability): 새로운 정보나 경험을 바탕으로 스스로 학습 및 발전
 - 반응성(Reactivity): 환경 변화에 반응하여 행동/판단을 조정
 - 주도성(Proactivity): 사용자 요청이 없어도 선제적 행동 수행

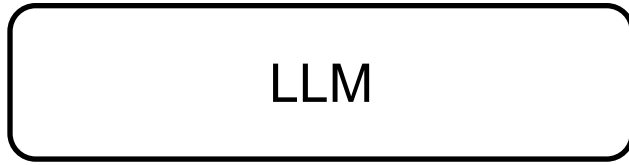


에이전트 동작 메커니즘

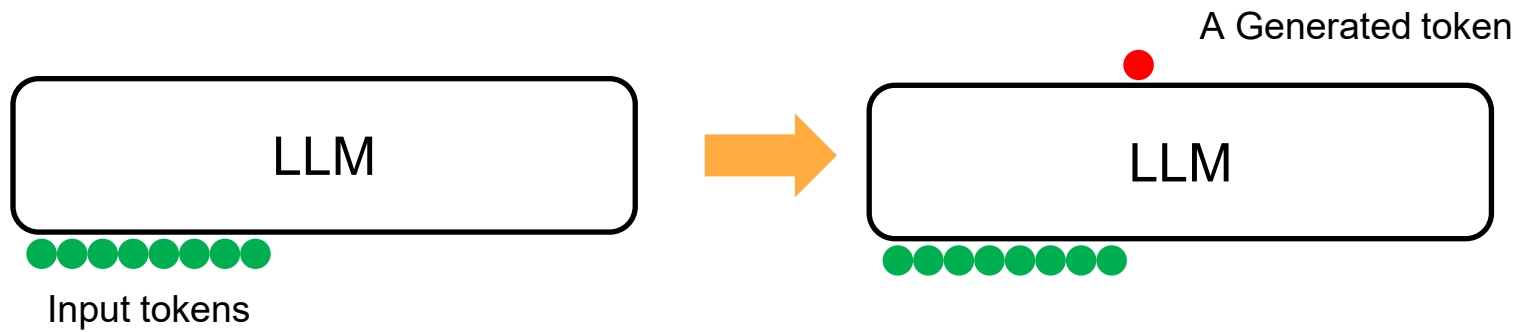
LLM과 컨텍스트

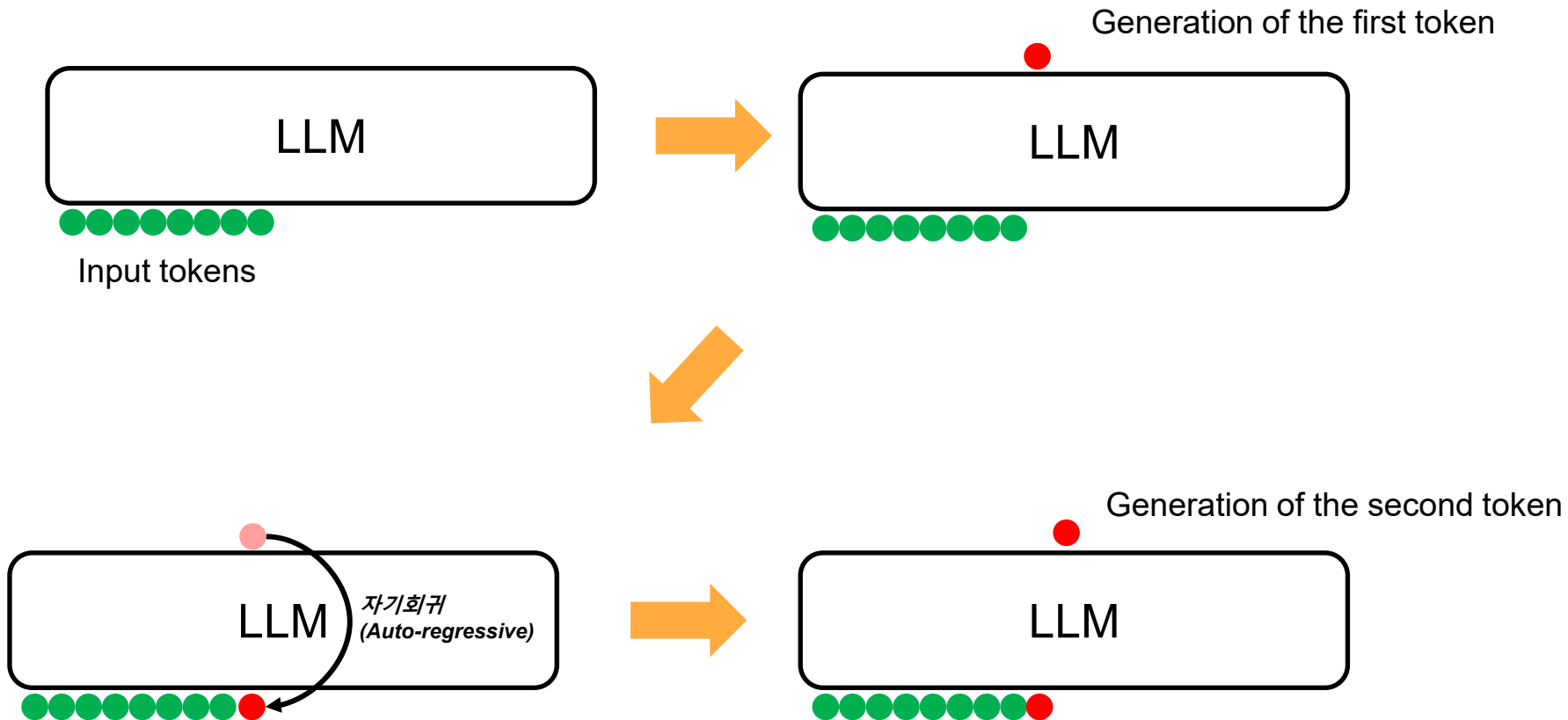
- 컨텍스트(Context)란 LLM이 입력 받은 모든 토큰화 된 데이터
 - 프롬프트(Prompt)는 AI의 행동을 유도하는 지시사항. 컨텍스트에 포함됨.

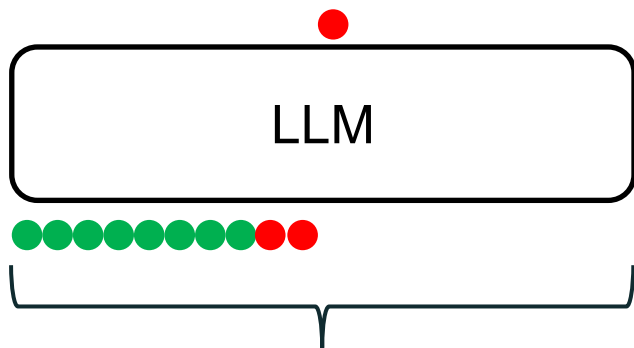




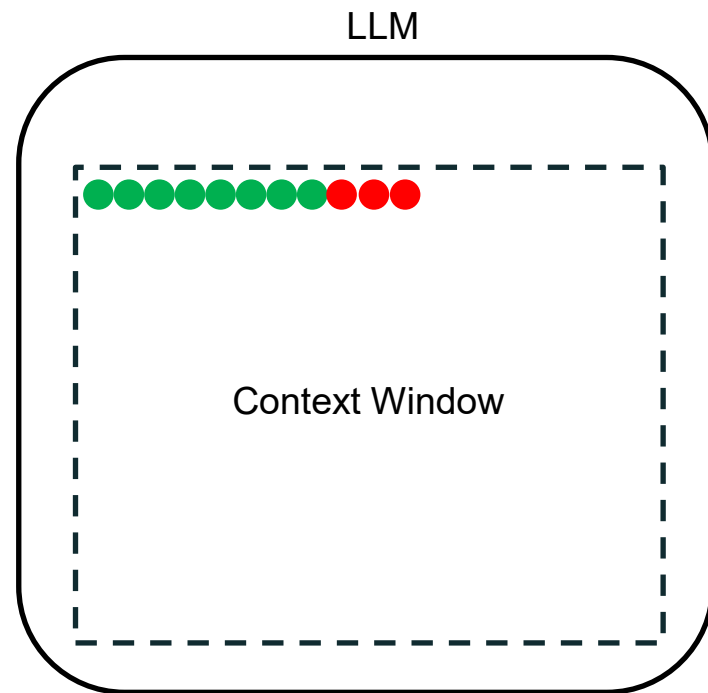
Input tokens



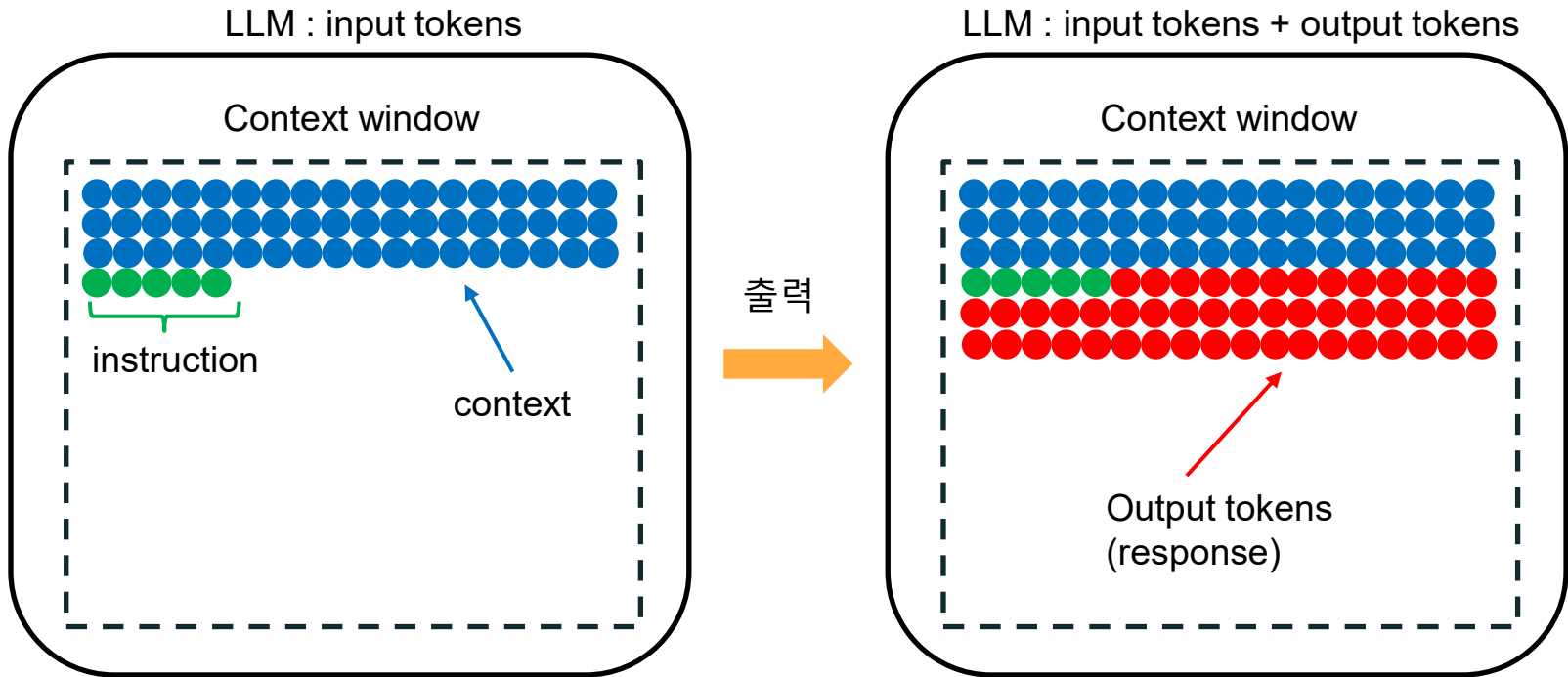




최대 입력 길이
(Context length)



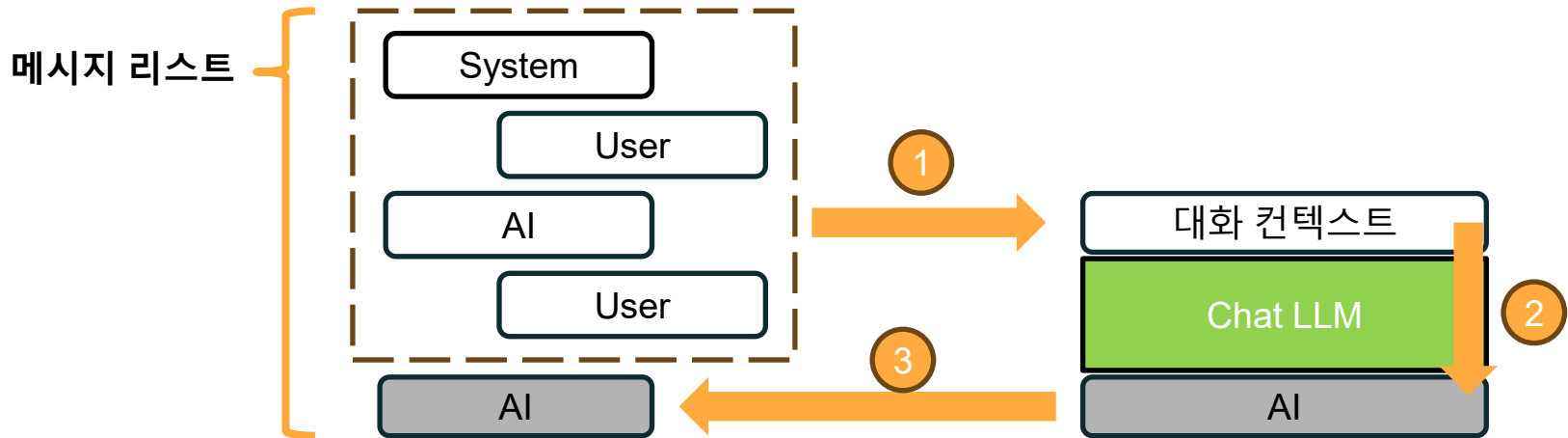
유한한 크기(context length)의 가상의
Context window를 상정 할 수 있음

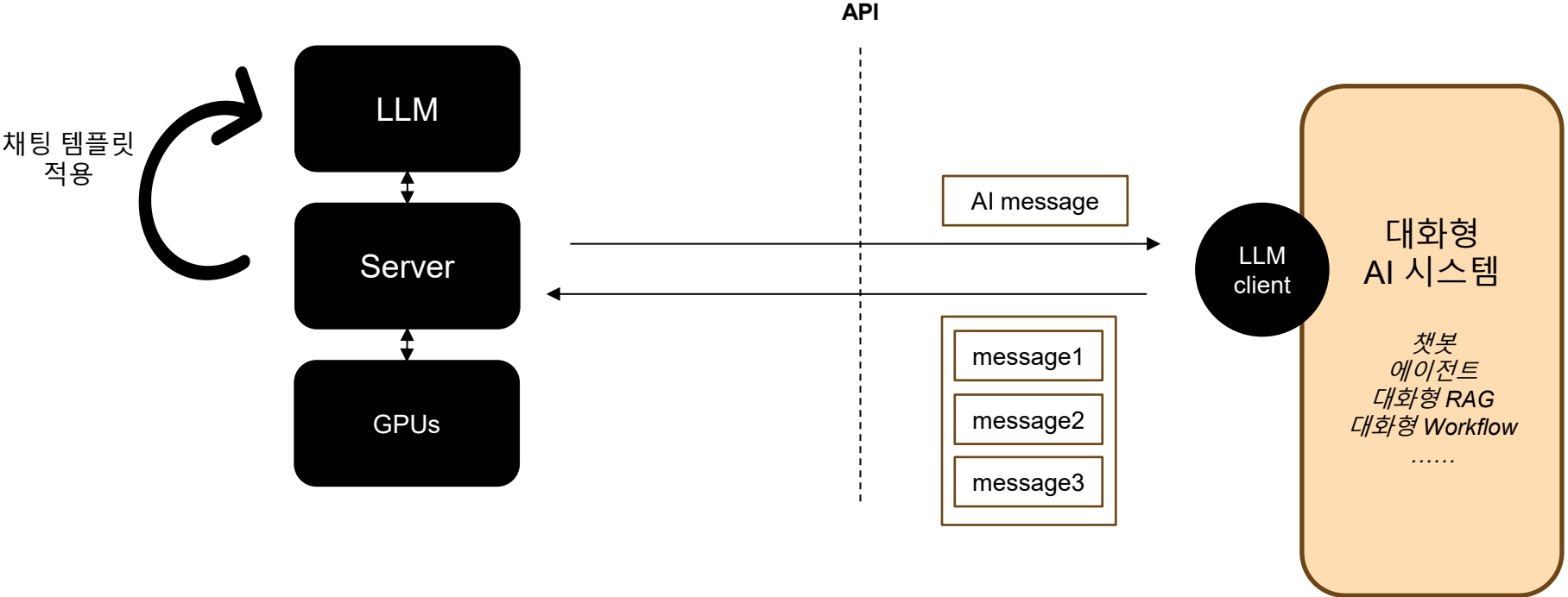


- LLM에 적절한 데이터를 적절한 시점에 Context Window에 입력하는 것이 목표
 - 이를 위한 시스템이 Chatbot, RAG, Agent와 같은 LLM application.

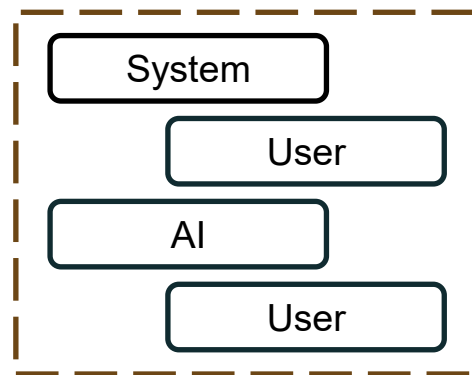
Chat LLM과 대화 컨텍스트

구분	설명	역할
System Message	대화의 규칙과 역할을 정의하는 메시지	AI의 성격, 말투, 목표를 설정
User Message	사용자의 질문이나 요청	AI가 반응해야 하는 유저의 입력(자연어 명령)
AI Message	LLM이 생성한 응답	사용자 입력과 시스템 지시를 바탕으로 산출된 출력





```
[{'role': 'system',  
  'content': ' You are an artificial intelligence assistant that answers in Korean.'},  
{ 'role': 'user', 'content': '제 이름은 김형욱입니다. 취미는 독서예요.'},  
{ 'role': 'asistant', 'content': '안녕하세요! 형욱. 좋은 취미예요.'},  
{ 'role': 'user', 'content': '제 취미를 더 잘 즐길 수 있는 방법이 있을까요?'}]
```



↓
모델의 템플릿으로 메시지들을 **대화 컨텍스트**로 변환.
(단일 텍스트)

```
[|user|]  
제 이름은 김형욱입니다. 취미는 독서예요.[|endofturn|]  
[|assistant|]  
안녕하세요! 형욱. 좋은 취미예요.[|endofturn|]  
[|user|]  
제 취미를 더 잘 즐길 수 있는 방법이 있을까요?[|endofturn|]  
[|assistant|]
```

- 학습 데이터를 통해 [|assistant|] 뒤에 자신(AI)의 답변이 위치하는 것을 패턴 학습했음.
- 앞선 대화 컨텍스트를 바탕으로 뒤에 자연스러운 답변 생성

[|user|]
제 이름은 김형욱입니다. 취미는 독서예요.[|endofturn|]
[|assistant|]
안녕하세요! 형욱. 좋은 취미예요.[|endofturn|]
[|user|]
제 취미를 더 잘 즐길 수 있는 방법이 있을까요?[|endofturn|]
[|assistant|]



토큰화

360, 560, 1050, 3730, 732, 50705, 3095, 10996, 375, 15563,
657, 12815, 2373, 740, 375, 361, 560, 359, 560, 3801,
1130, 5847, 362, 2138, 3095, 375, 1104, 732, 15563, 2373,
740, 375, 361, 560, 360, 560, 1050, 15563, 4605, 1047,
1353, 10215, 868, 773, 657, 2818, 634, 773, 696, 4148,
392, 361, 560, 359

360, 560, 1050, 3730, 732, 50705, 3095, 10996, 375, 15563,
657, 12815, 2373, 740, 375, 361, 560, 359, 560, 3801,
1130, 5847, 362, 2138, 3095, 375, 1104, 732, 15563, 2373,
740, 375, 361, 560, 360, 560, 1050, 15563, 4605, 1047,
1353, 10215, 868, 773, 657, 2818, 634, 773, 696, 4148,
392, 361, 560, 359

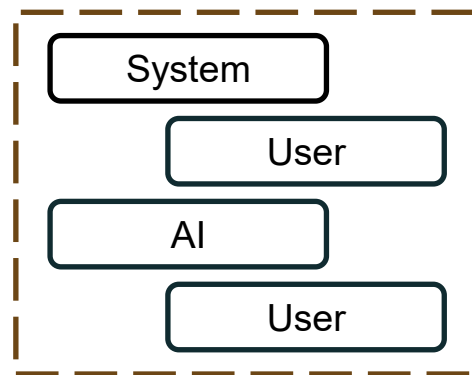


LLM의 응답 생성

360, 560, 1050, 3730, 732, 50705, 3095, 10996, 375, 15563,
657, 12815, 2373, 740, 375, 361, 560, 359, 560, 3801,
1130, 5847, 362, 2138, 3095, 375, 1104, 732, 15563, 2373,
740, 375, 361, 560, 360, 560, 1050, 15563, 4605, 1047,
1353, 10215, 868, 773, 657, 2818, 634, 773, 696, 4148,
392, 361, 560, 359, 560, 560, 1023, 373, 12815, 2373,
2047, 4288, 634, 1990, 6001, 740, 375, 1058, 3730, 732,
50705, 3095, 634, 3885, 691, 5847, 375, 560, 560, 57553,

.....

```
[{'role': 'system',  
  'content': ' You are an artificial intelligence assistant that answers in Korean.'},  
{ 'role': 'user', 'content': '제 이름은 김형욱입니다. 취미는 독서예요.'},  
{ 'role': 'asistant', 'content': '안녕하세요! 형욱. 좋은 취미예요.'},  
{ 'role': 'user', 'content': '제 취미를 더 잘 즐길 수 있는 방법이 있을까요?'}]
```



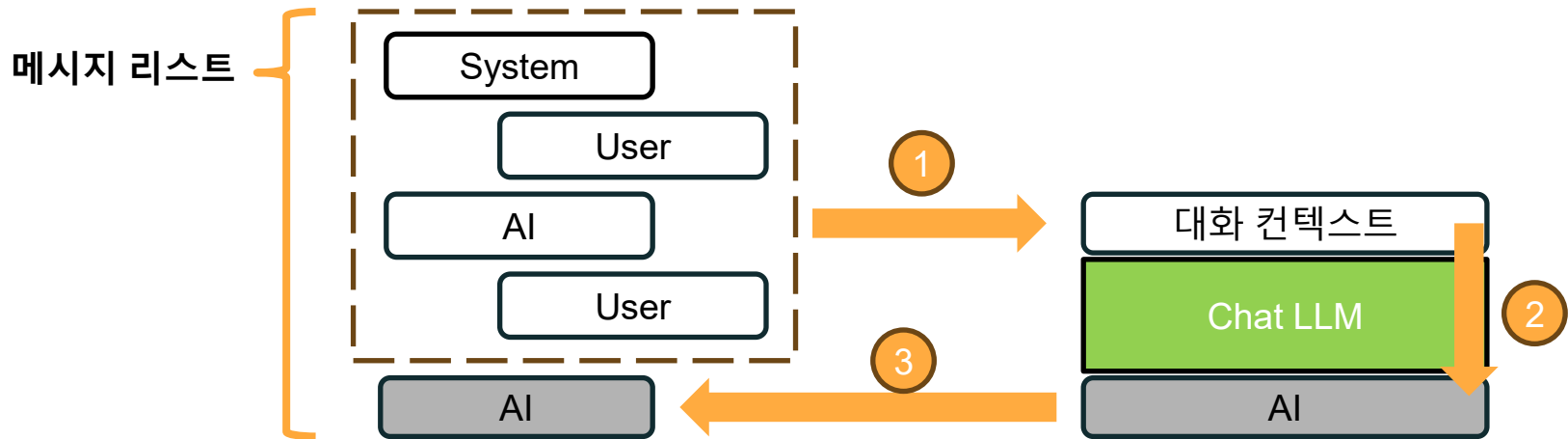
↓
모델의 템플릿으로 메시지들을 **대화 컨텍스트**로 변환.
(단일 텍스트)

```
[|user|]  
제 이름은 김형욱입니다. 취미는 독서예요.[|endofturn|]  
[|assistant|]  
안녕하세요! 형욱. 좋은 취미예요.[|endofturn|]  
[|user|]  
제 취미를 더 잘 즐길 수 있는 방법이 있을까요?[|endofturn|]  
[|assistant|]
```

이 부분부터 AI의 출력 토큰 생성 : AI의 답변

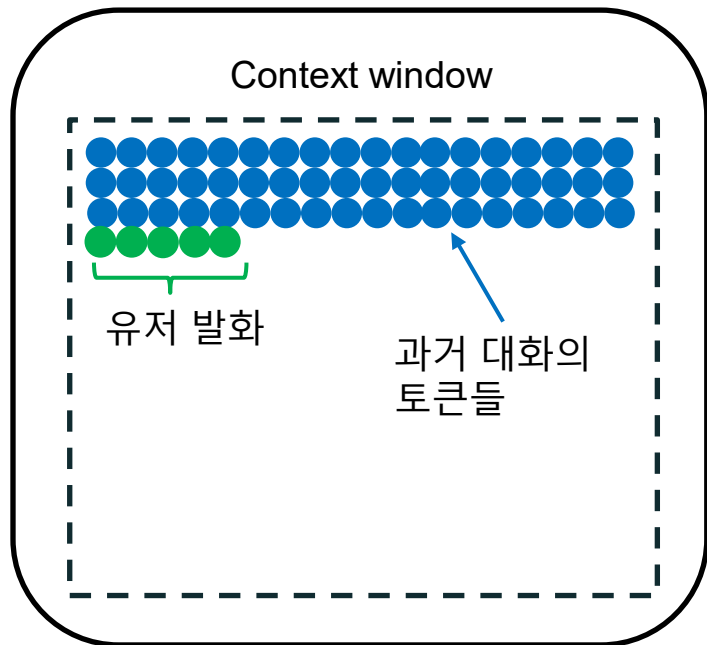
AI

- 학습을 통해 [|assistant|] 뒤에 자신(AI)의 답변이 온다는 걸 패턴 학습했음.
- 앞선 대화 컨텍스트를 바탕으로 뒤에 자연스러운 답변 생성



- User 메시지, AI 메시지는 반드시 메시지 리스트에 축적
- 대화 히스토리를 유지하는 것은 다중 회차 대화를 위해 반드시 필요

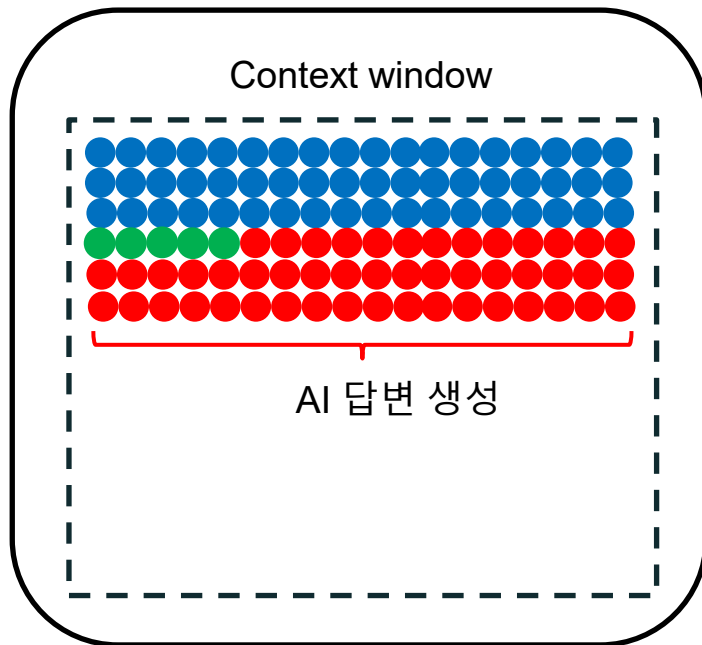
LLM

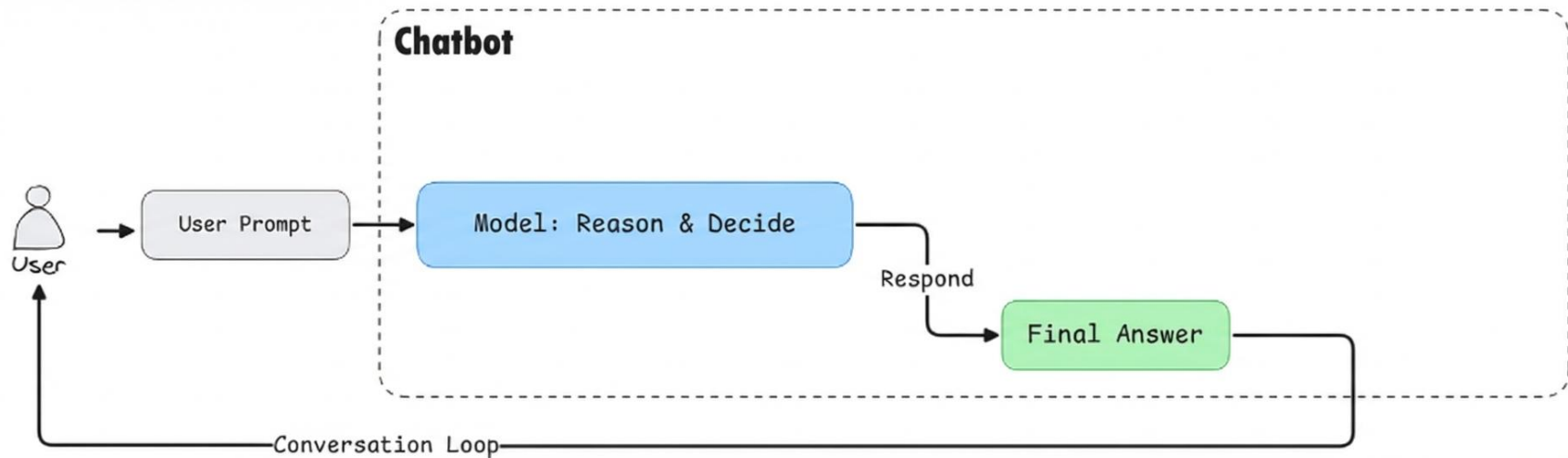


출력



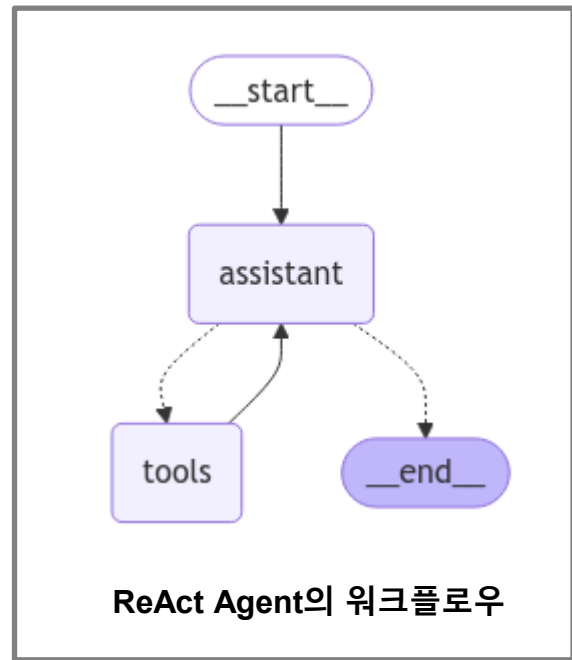
LLM





ReAct : Reason-then Act Agent

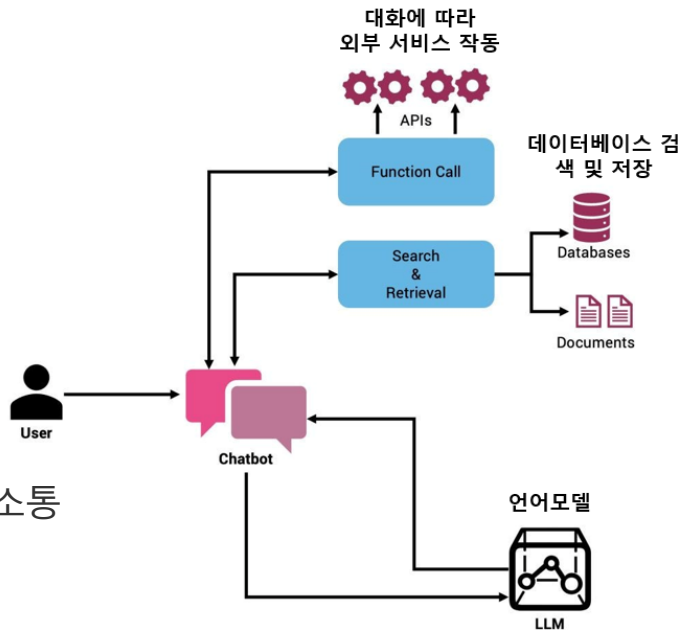
- 스스로 판단하여 행동할 수 있는 대화형 AI 에이전트
- 에이전트의 행동이란?
 - 에이전트가 행동으로서 실행할 수 있도록 연동된 기능
 - 인지, 추론, 실행의 프로세스를 통해 문제 해결 능력 부여
 - 실제로는 **Tool**이라는 것을 **실행**하여 행동
- Tool 실행 프로세스
 - Step 1. LLM가 Tool 호출(Tool 이름, 인자)
 - Step 2. Tool 실행
 - Step 3. 실행 결과를 메시지에 담아서 LLM에게 전달
 - Step 4. 필요 시 재귀적 Tool 실행



ReAct 대화 컨텍스트

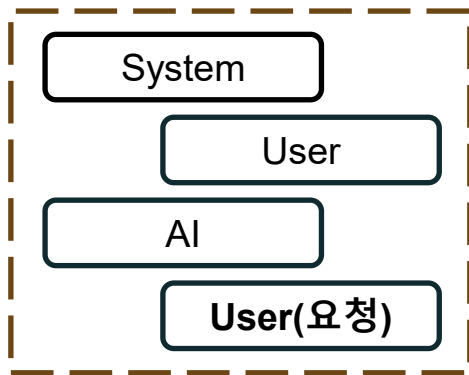
- 사람과는 언어로, 시스템과는 도구로 대화
 - 사용자와 상호작용
 - ✓ 채팅 인터페이스를 통해 자연어로 소통
 - 시스템과 상호작용
 - ✓ 도구를 통해 프로그램·데이터베이스·서버·시스템 등과 소통

구분	상호작용 방식	예시
User ↔ Agent	자연어 대화 (Chat UI)	"요약해줘", "파일 분석해줘"
Agent ↔ System	Tool 호출 (Function / API)	WebSearch, Database Query, API Call

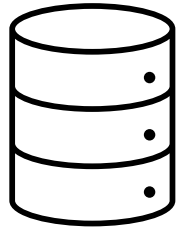


Agent와 도구 컨텍스트

구분	설명	역할
Tool Message	도구 실행 결과를 전달하는 메시지	LLM이 호출한 함수·API의 실제 실행 결과



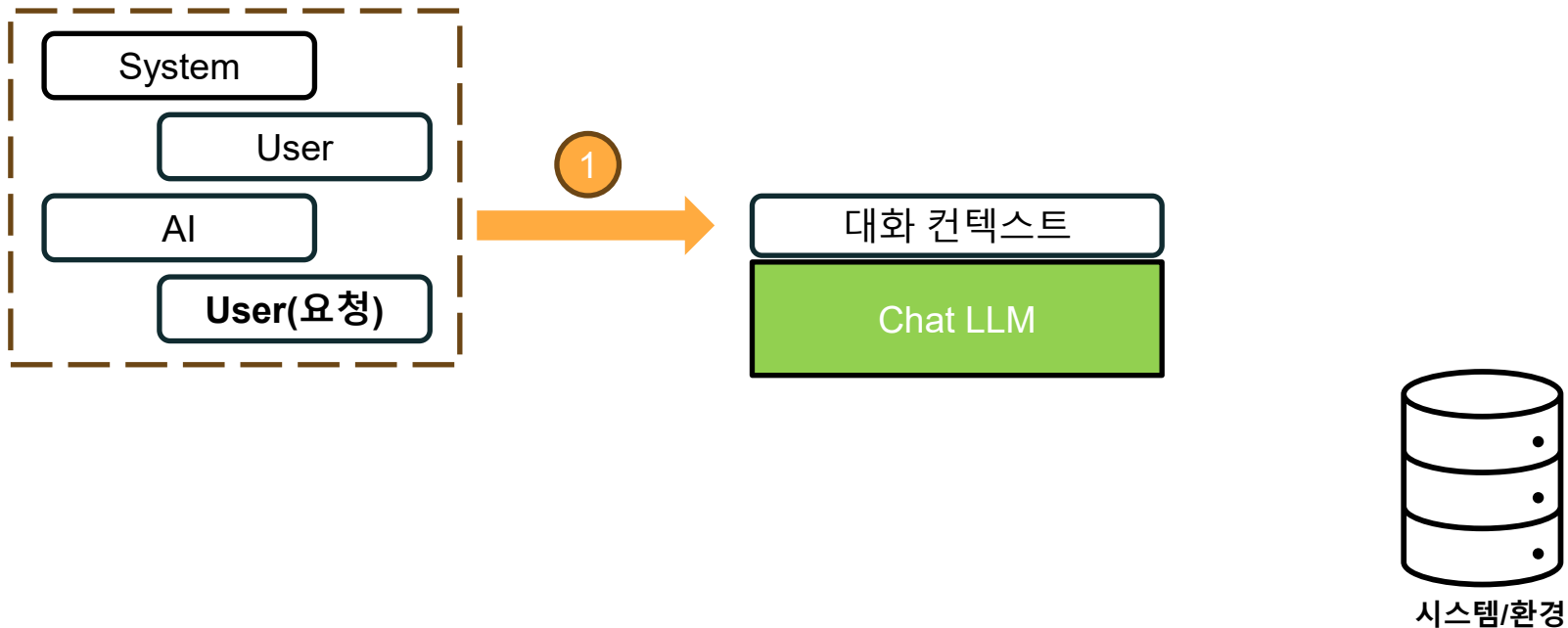
Chat LLM



시스템/환경

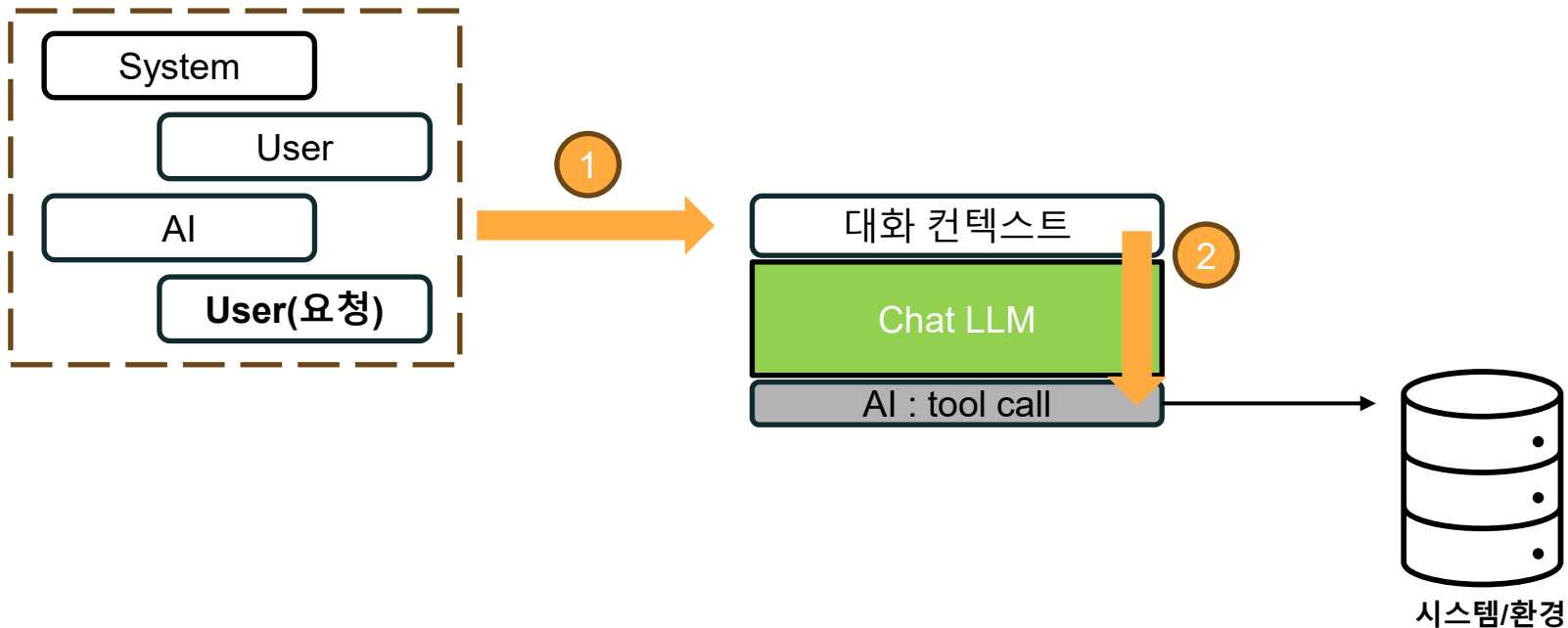
Agent와 도구 컨텍스트

구분	설명	역할
Tool Message	도구 실행 결과를 전달하는 메시지	LLM이 호출한 함수·API의 실제 실행 결과



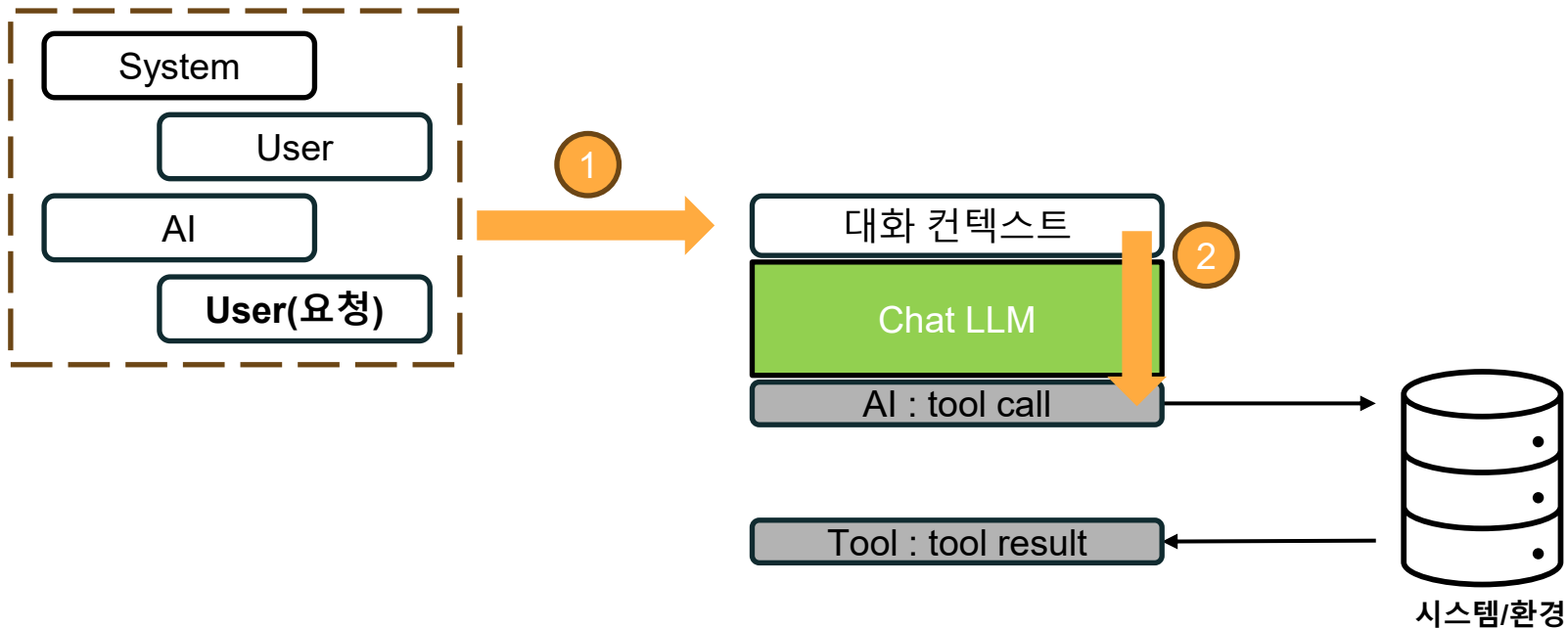
Agent와 도구 컨텍스트

구분	설명	역할
Tool Message	도구 실행 결과를 전달하는 메시지	LLM이 호출한 함수·API의 실제 실행 결과



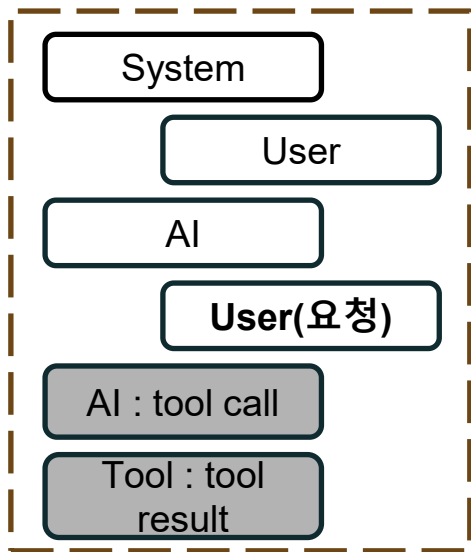
Agent와 도구 컨텍스트

구분	설명	역할
Tool Message	도구 실행 결과를 전달하는 메시지	LLM이 호출한 함수·API의 실제 실행 결과

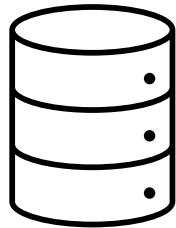


Agent와 도구 컨텍스트

구분	설명	역할
Tool Message	도구 실행 결과를 전달하는 메시지	LLM이 호출한 함수·API의 실제 실행 결과



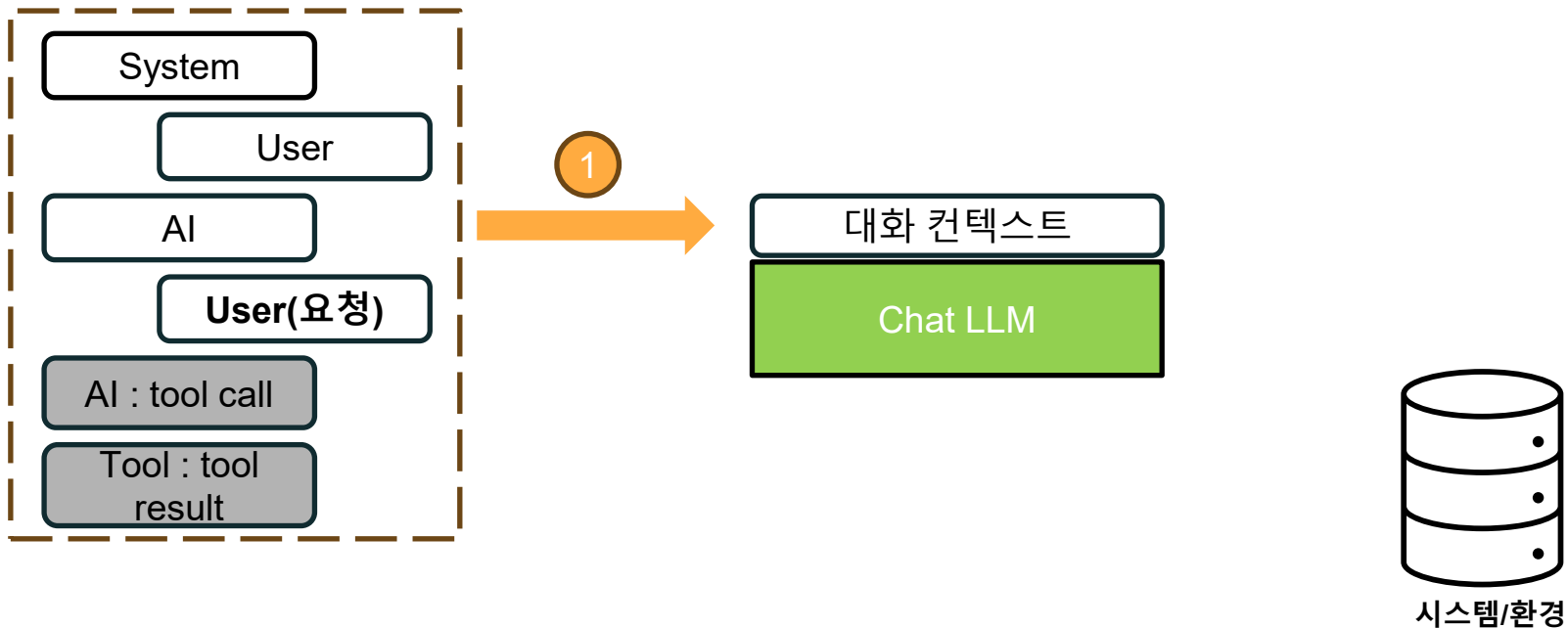
Chat LLM



시스템/환경

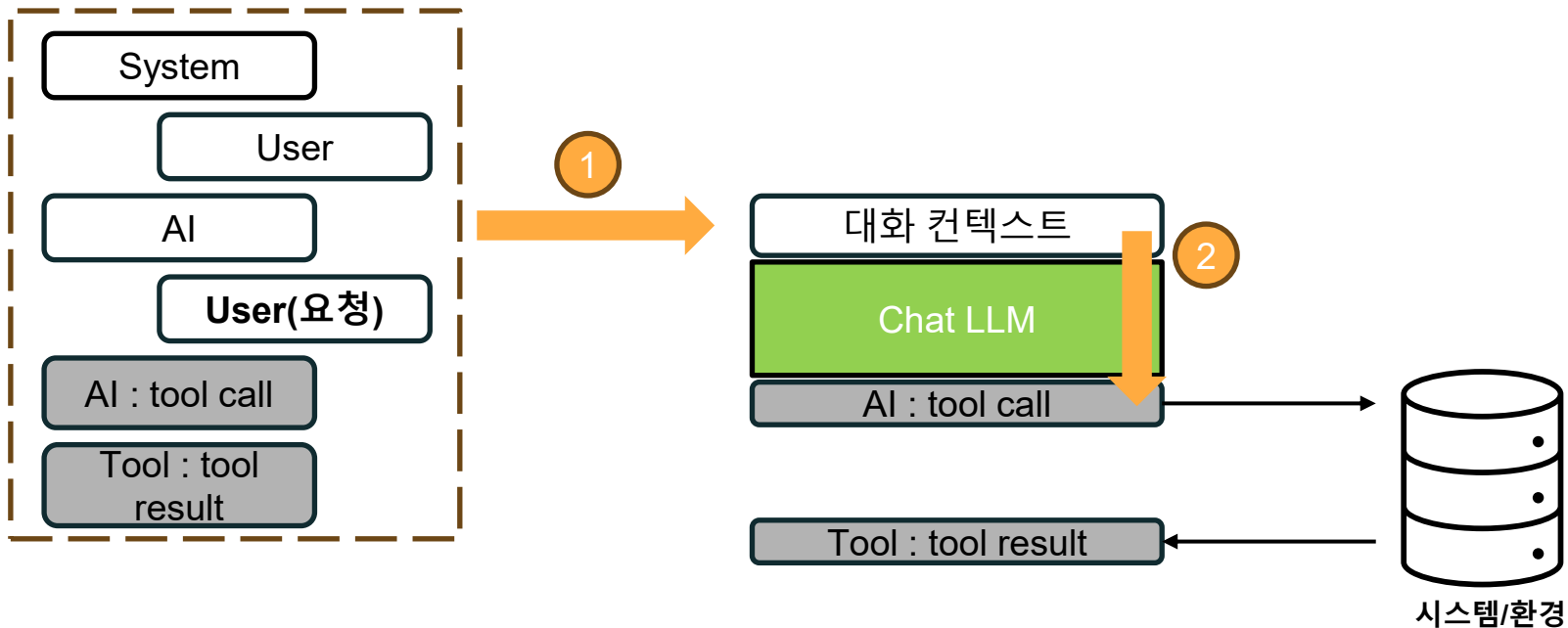
Agent와 도구 컨텍스트

구분	설명	역할
Tool Message	도구 실행 결과를 전달하는 메시지	LLM이 호출한 함수·API의 실제 실행 결과



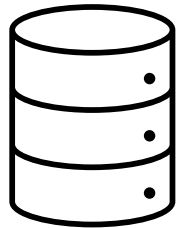
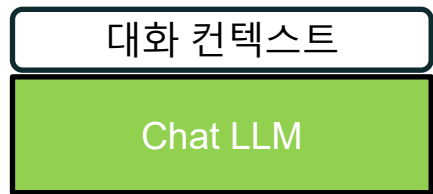
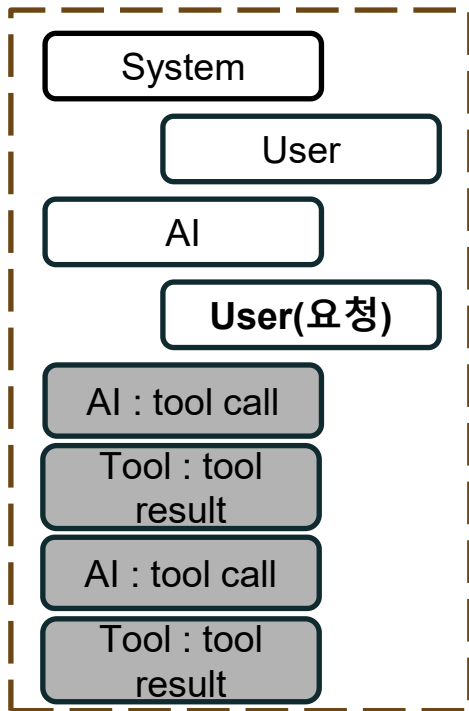
Agent와 도구 컨텍스트

구분	설명	역할
Tool Message	도구 실행 결과를 전달하는 메시지	LLM이 호출한 함수·API의 실제 실행 결과



Agent와 도구 컨텍스트

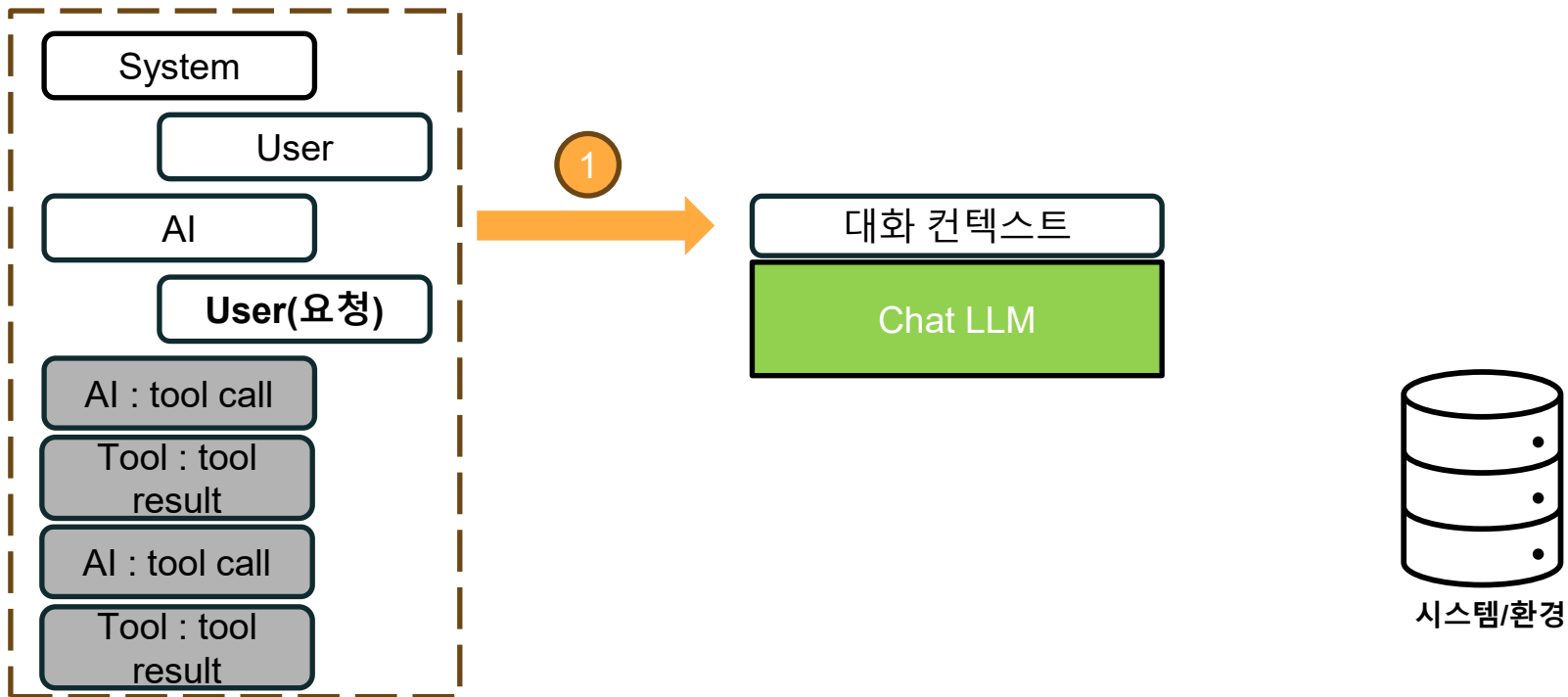
구분	설명	역할
Tool Message	도구 실행 결과를 전달하는 메시지	LLM이 호출한 함수·API의 실제 실행 결과



시스템/환경

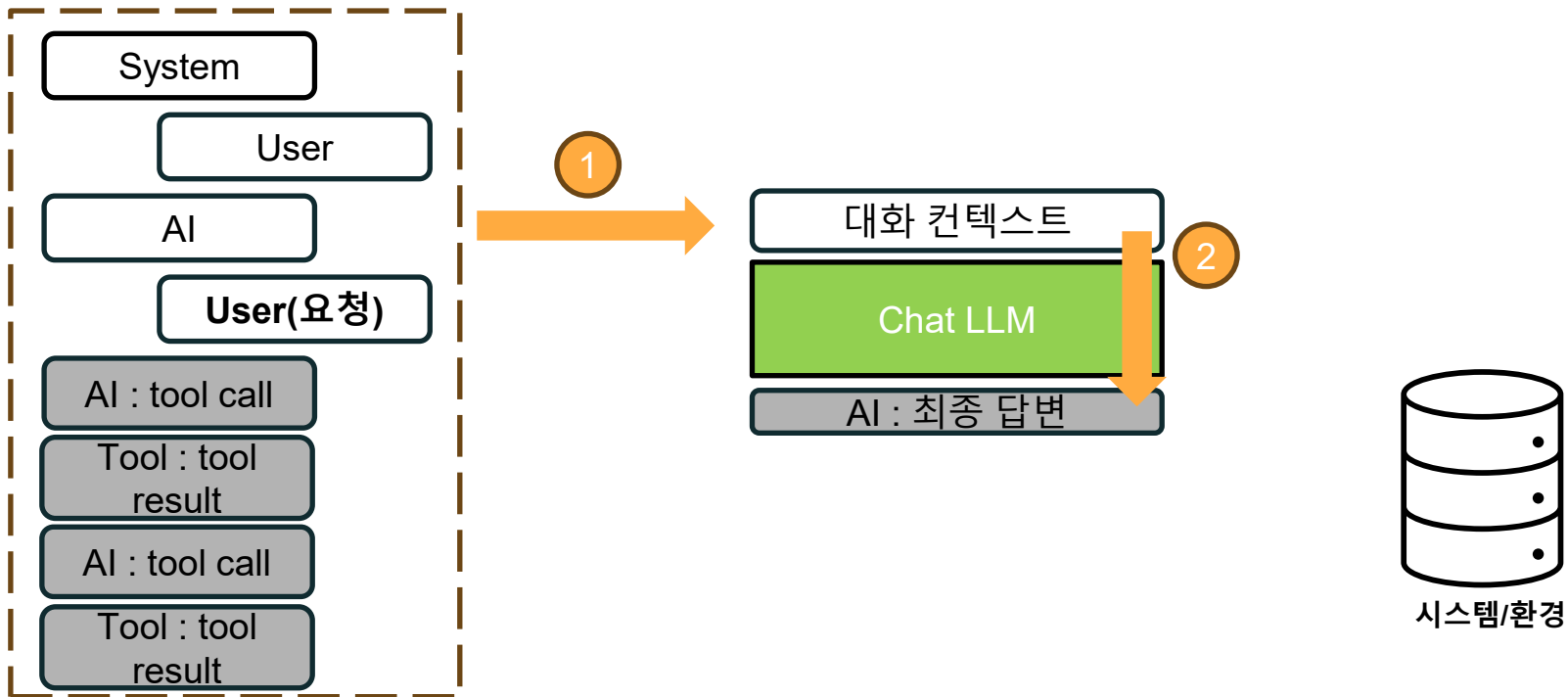
Agent와 도구 컨텍스트

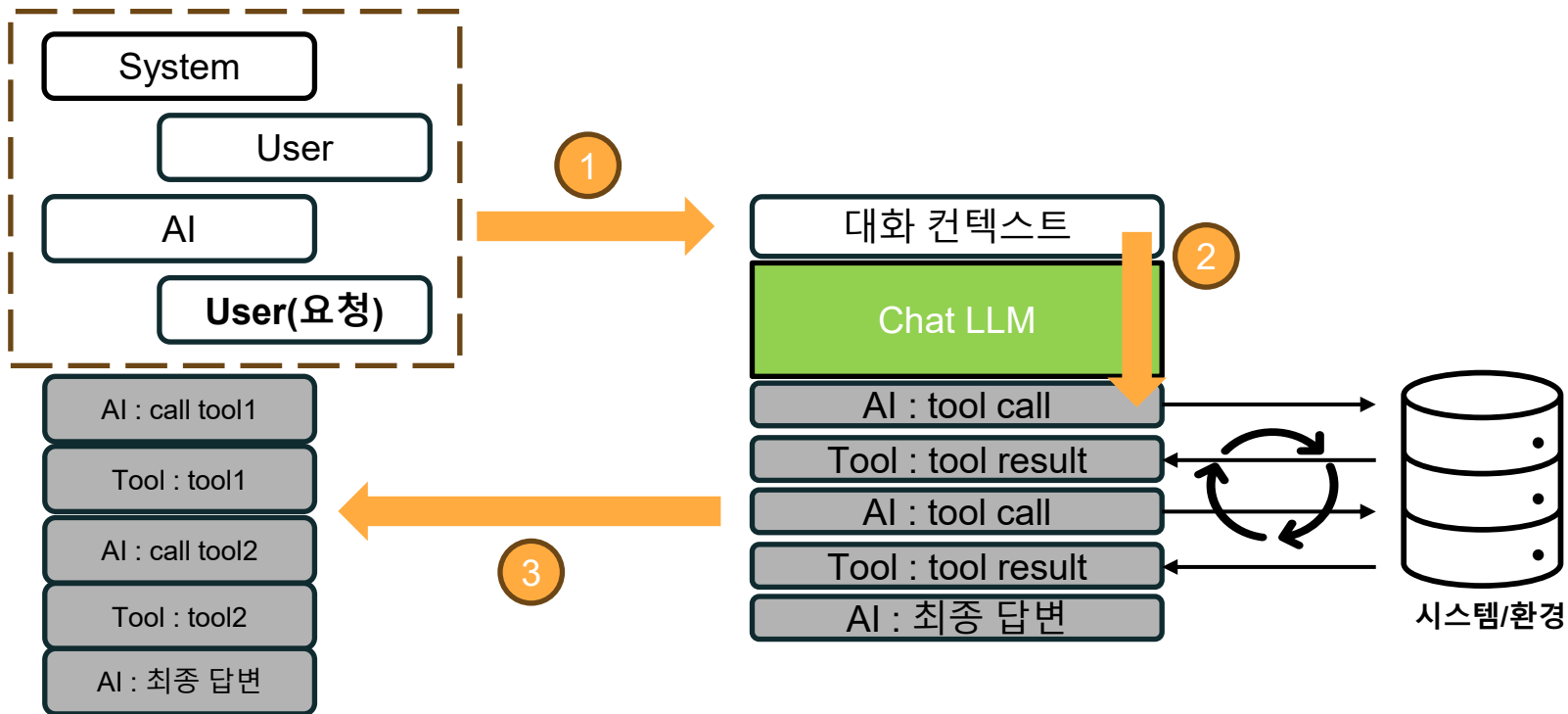
구분	설명	역할
Tool Message	도구 실행 결과를 전달하는 메시지	LLM이 호출한 함수·API의 실제 실행 결과



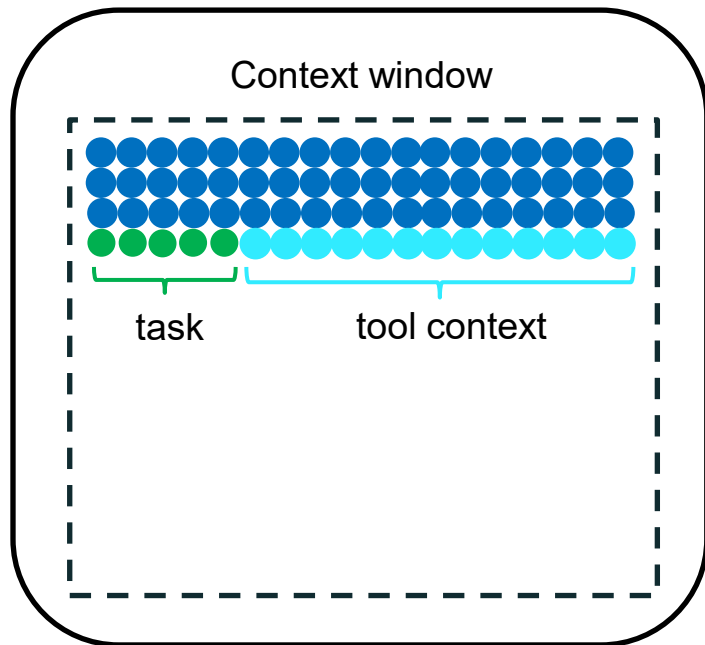
Agent와 도구 컨텍스트

구분	설명	역할
Tool Message	도구 실행 결과를 전달하는 메시지	LLM이 호출한 함수·API의 실제 실행 결과

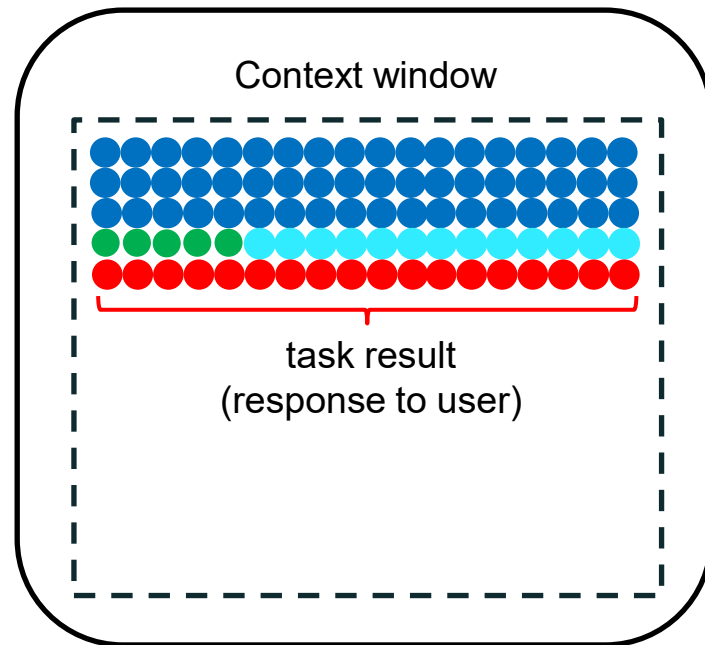


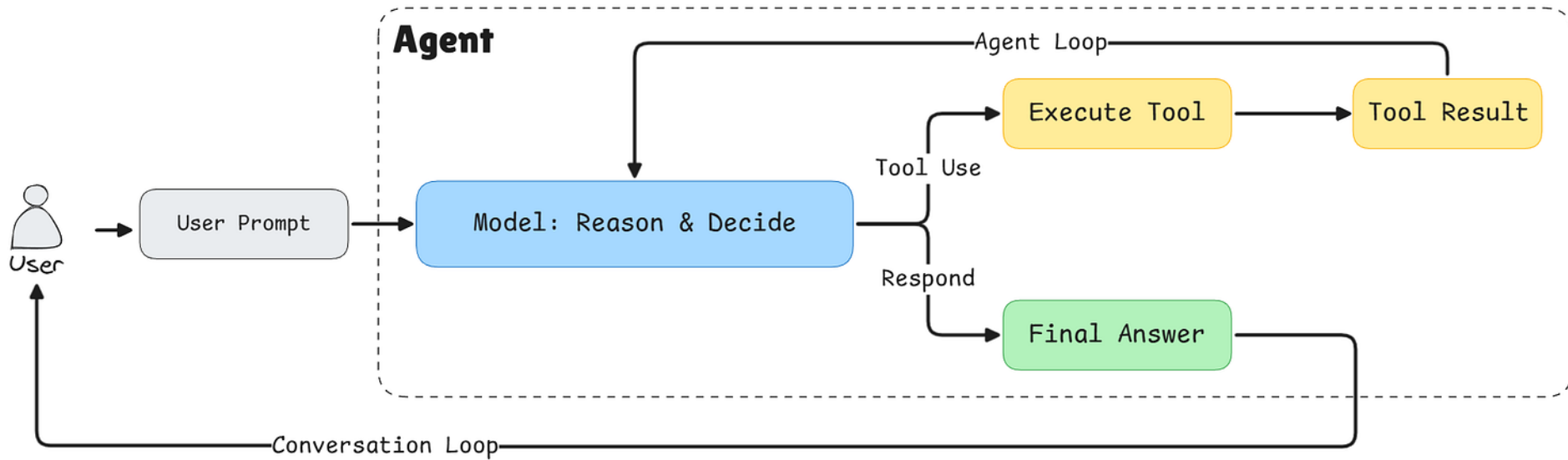


LLM



출력

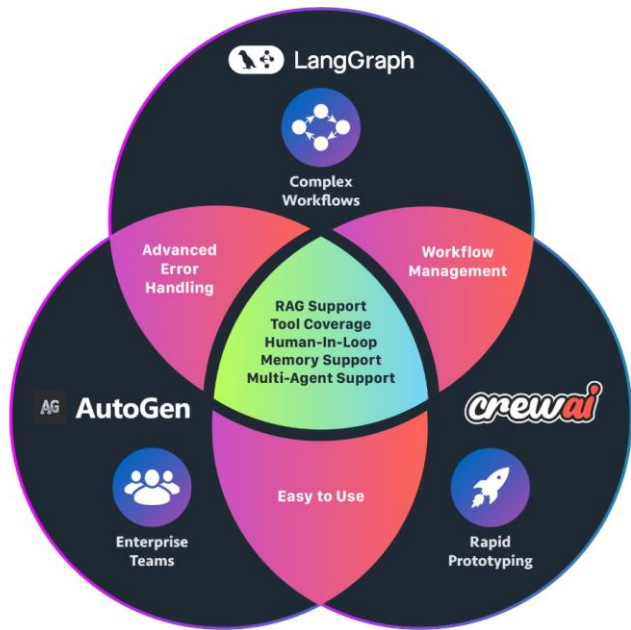




왜 LangGraph로 배울까?

- 다른 프레임워크 대비 에이전트를 정교하게 제어 가능
 - 상태 기반 : 입력·출력·컨텍스트 등을 상태로 관리하여, 작업·결과가 추적되고 재현 가능한 구조 제공
 - 유연성/확장성 : 에이전트의 동작을 노드/상태 레벨로 세밀하게 설계, 멀티 에이전트 시스템 구축 가능
 - 워크플로우와 통합 : 반복적이거나 정확성이 요구되는 작업을 하위 워크플로우로 구성하여 에이전트와 자연스럽게 통합 가능
- 에이전트를 공부하기 좋음
 - LangChain 생태계의 방대한 유저 커뮤니티, 기술문서와 구현 사례
 - 에이전트의 작동 원리를 저수준에서 구현/테스트 가능

주요 Agent 개발 프레임워크 비교

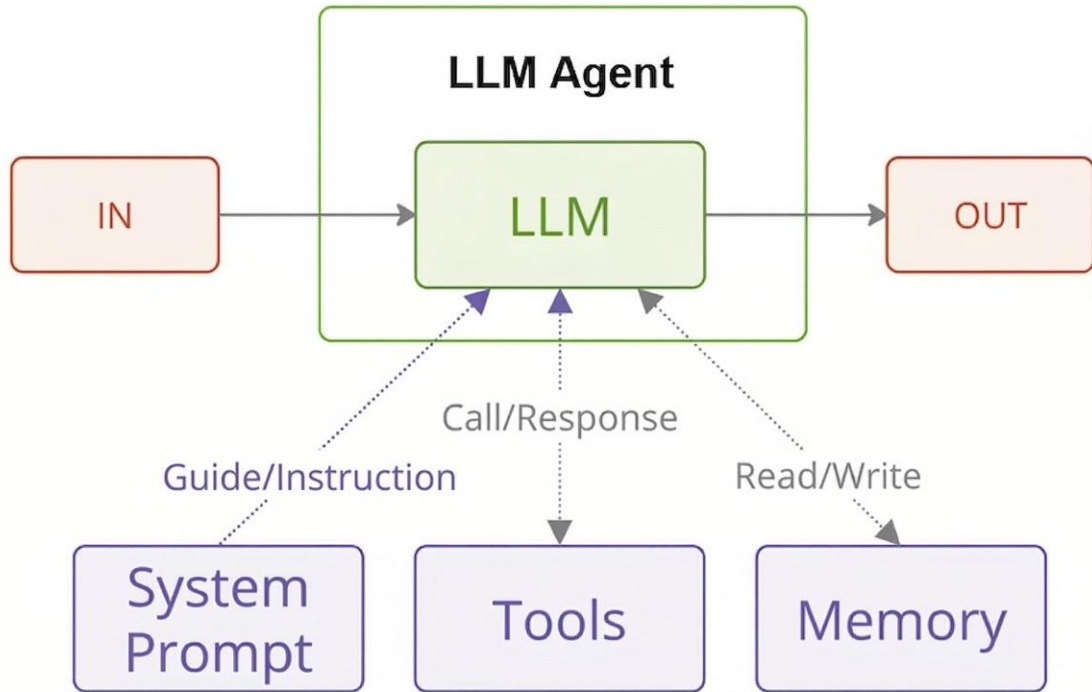


프레임워크	주요 특징	장단점
LangGraph	그래프 기반 워크플로우/에이전트	• 복잡한 다단계 프로세스 제어 가능 • 초기 학습 난이도 있음
AutoGen	엔터프라이즈급 AI 에이전트 프레임워크	• 높은 에이전트 추상성 • 정교한 제어 한계
CrewAI	빠른 프로토타이핑 및 쉬운 설정	• 빠른 개발 및 테스트 가능 • 정교한 제어 한계

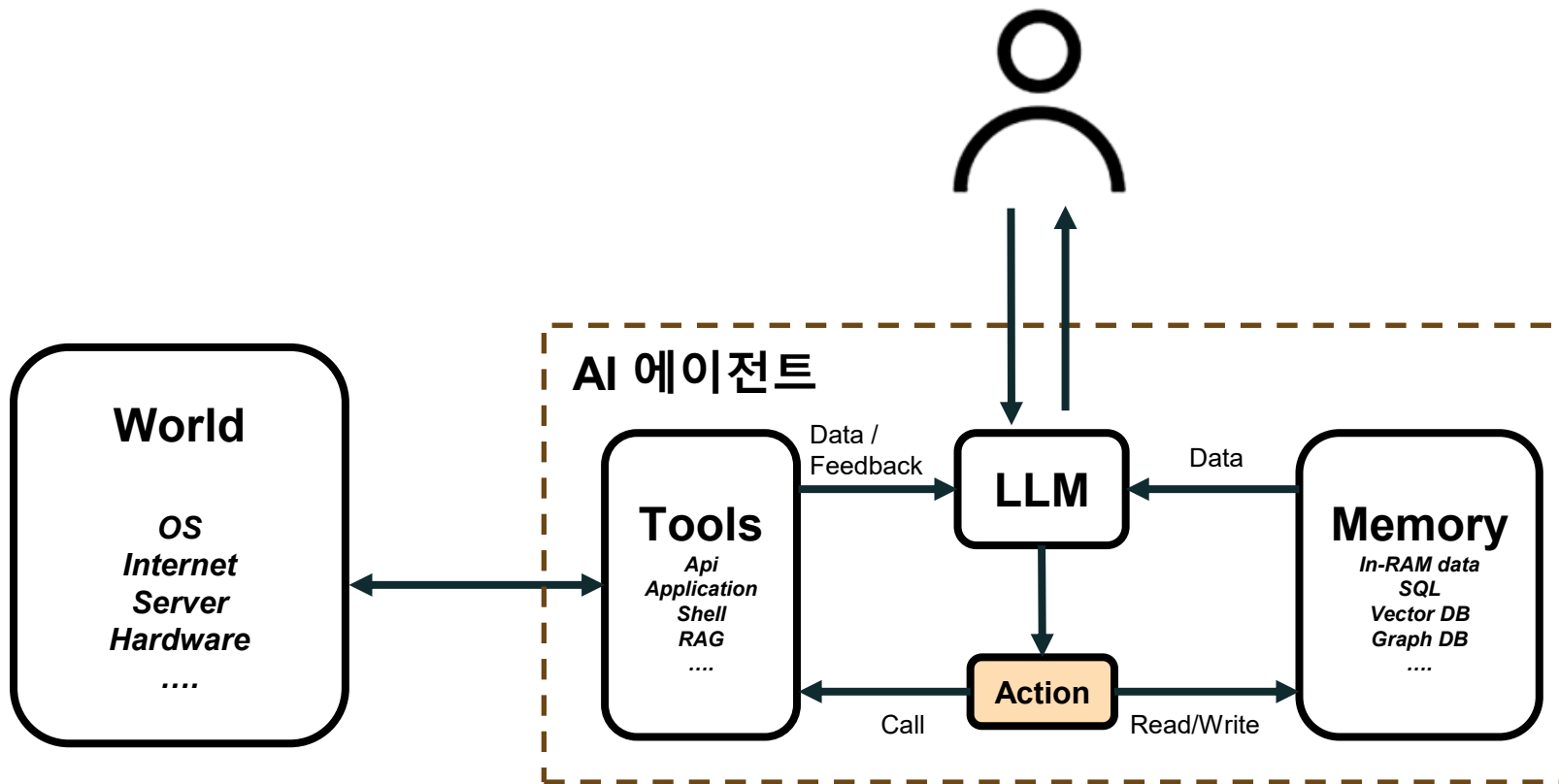
LangGraph ↔ AutoGen: 호환 가능
(LangGraph로 흐름을 관리하고, AutoGen을 실행 엔진으로 활용 가능)

에이전트 설계

에이전트의 핵심 3요소



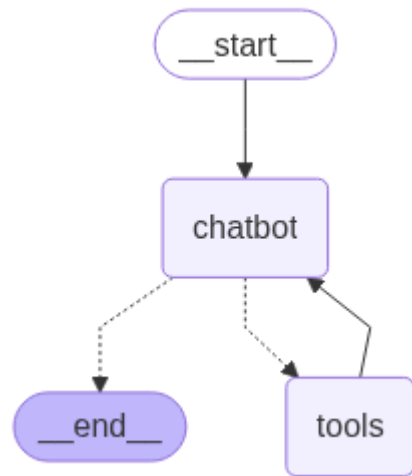
에이전트의 구조



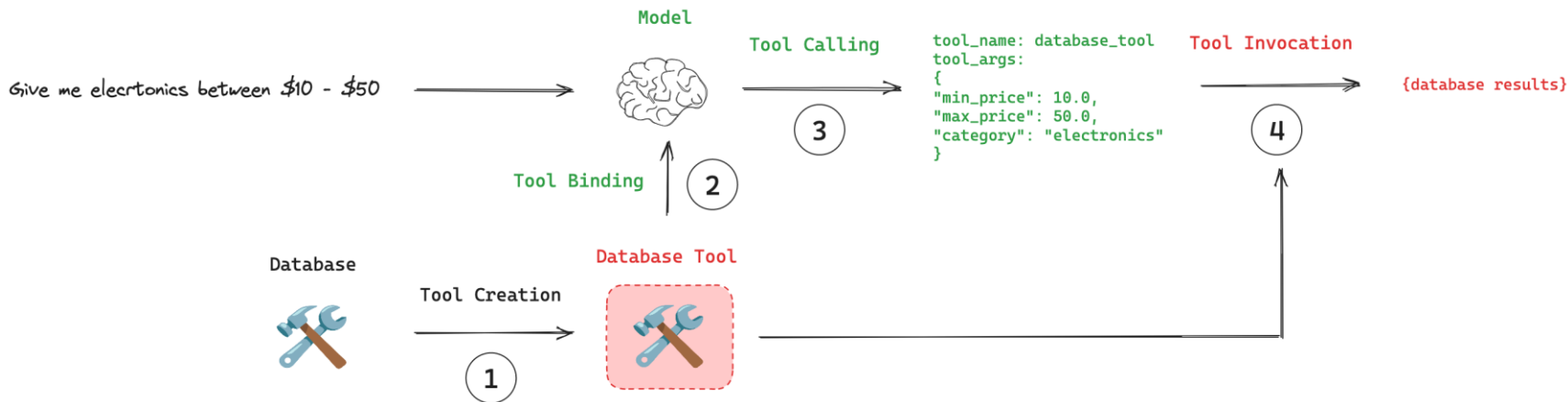
(1) Tool

- Tool이란?

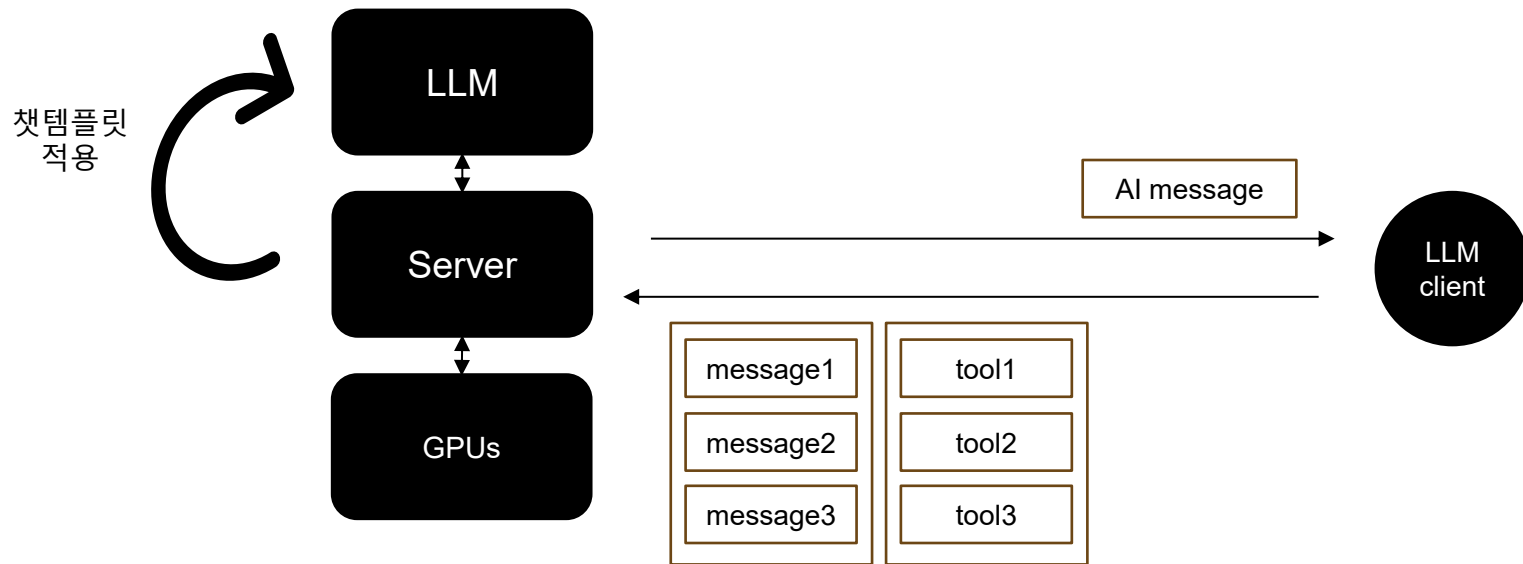
- 에이전트가 행동으로서 실행할 수 있는 기능
 - ✓ 함수, 검색, 외부 API 호출, DB 데이터 요청 등
 - ✓ 즉 Tool은 에이전트의 연장으로, 에이전트가 수행할 수 있는 전문적 업!
- 에이전트는 **응답 생성** or **Tool** 중 결정
 - ✓ 에이전트의 LLM은 컨텍스트를 바탕으로 결정을 추론
- Tool이 실행되는 과정
 - ✓ (**Tool call**) Tool name과 arguments에 대한 구조화된 응답 생성
 - ✓ (**Tool invocation**) Tool 노드에서 실행 후 결과를 LLM에게 전달



LangGraph Agent graph
(ReAct pattern)



- LangGraph에서 Tool은 LLM이 복잡한 작업을 수행할 수 있도록 돕는 핵심 구성 요소
 - (1) Tool creation : 에이전트에게 필요한 도구를 정의
 - (2) Tool binding : 에이전트가 쓸 도구를 인식시킴
 - (3) Tool calling : 컨텍스트를 바탕으로 에이전트가 Tool을 호출
 - (4) Tool invocation : Tool이 실행됨. 결과는 Tool Message에 담겨 대화 컨텍스트에 포함



Tool binding LLM의 내부 컨텍스트

```
<|im_start|>system
당신은 알리바바에 의해 개발된 Qwen 모델입니다. 당신은 한국어로 답변하는 친절한 어시스턴트입니다.

Current Date: 2025-06-18
```

System instruction

Tools

Tool-aware prompting via binding

You may call one or more functions to assist with the user query.

You are provided with function signatures within <tools></tools> XML tags:

```
<tools>
{"type": "function", "function": {"name": "get_current_temperature", "description": "특정 위치의 현재 기온을 가져옵니다.", "parameters": {"type": "object", "properties": {"location": {"type": "string", "description": "기온을 조회할 위치입니다. 예: \"Seoul, South Korea\".}, "unit": {"type": "string", "description": "반환할 기온의 단위입니다. 기본값은 \"Celsius\". 선택지: [\"Celsius\", \"Fahrenheit\"].}}, "required": ["location"]}}}
{"type": "function", "function": {"name": "get_temperature_date", "description": "특정 위치와 날짜의 기온을 가져옵니다.", "parameters": {"type": "object", "properties": {"location": {"type": "string", "description": "기온을 조회할 위치입니다. 예: \"Seoul, South Korea\".}, "date": {"type": "string", "description": "기온을 조회할 날짜입니다. 형식: \"YYYY-MM-DD\".}, "unit": {"type": "string", "description": "반환할 기온의 단위입니다. 기본값은 \"Celsius\". 선택지: [\"Celsius\", \"Fahrenheit\"].}}, "required": ["location", "date"]}}}
</tools>
```

For each function call, return a json object with function name and arguments within <tool_call></tool_call> XML tags:

```
<tool_call>
{"name": "<function-name>", "arguments": <args-json-object>}
</tool_call><|im_end|>
```

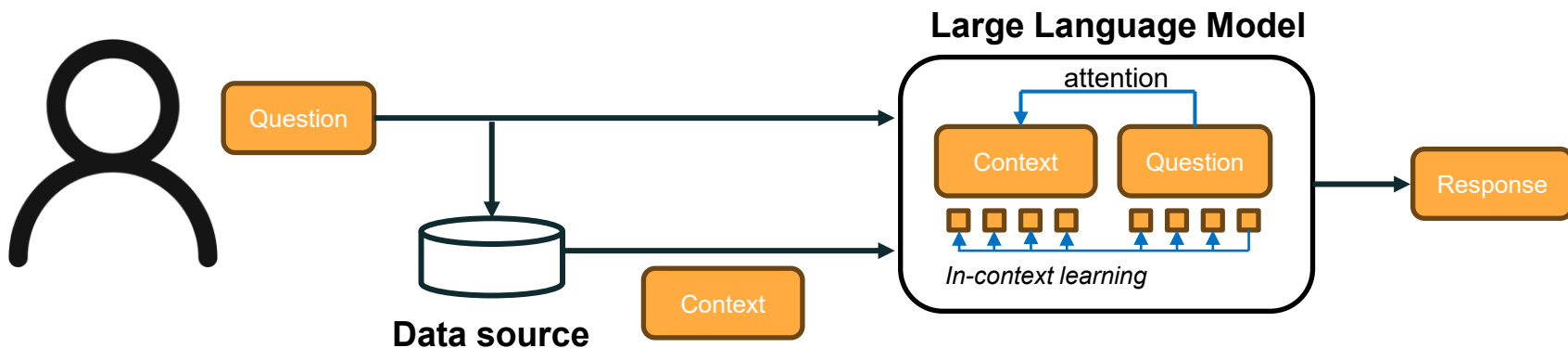
Enforcing specific tool-call formats
For precise tool invocation

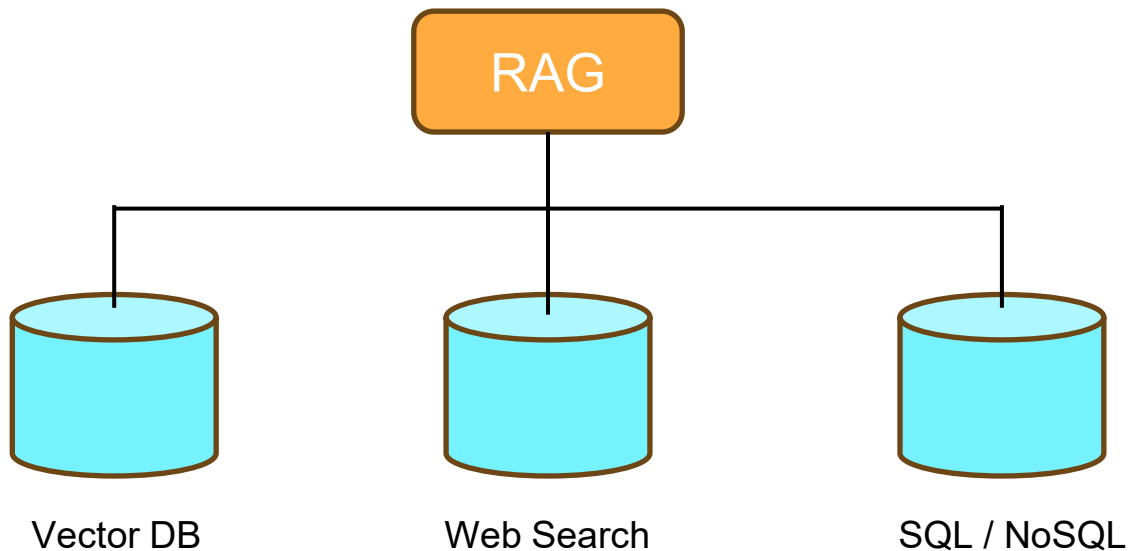
```
<|im_start|>user
서울의 온도는 몇 도죠? 내일은 어떤가요?<|im_end|>
<|im_start|>assistant
```

User & AI messages (대화 컨텍스트)

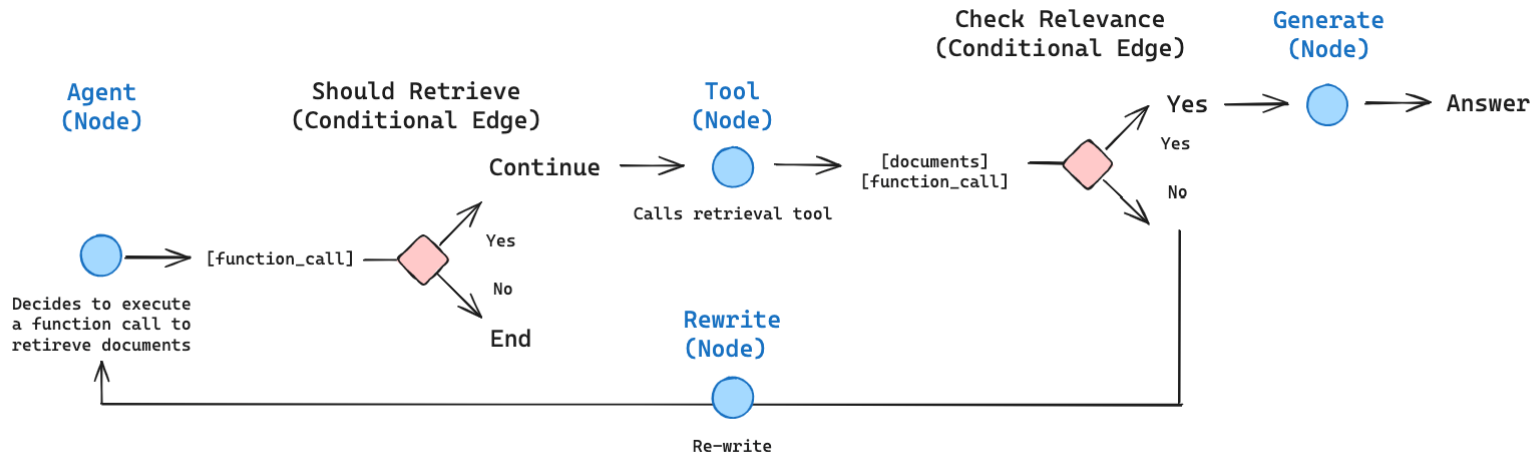
Retrieval

- LLM에 검색을 통해 컨텍스트를 제공하는 기법
 - LLM 단독 응답이 아닌, 검색 시스템과 결합하여 신뢰도 높은 정보를 제공
- LLM 자체의 지식에 의존하지 않고, **외부 문서를 검색**하여 응답 생성
 - 웹 문서, PDF, DB, 그래프 등 다양한 데이터 소스를 연결하여 활용





- RAG에서 사용할 수 있는 데이터 소스
 - Vector DB : 텍스트를 임베딩 벡터로 변환하고, 벡터화된 쿼리를 통해 semantic search
 - Web search : 실시간으로 인터넷을 검색해서 결과를 가져오고, 이를 LLM 입력 컨텍스트에 넣는 방식
 - Database : 일반 DB에서 키워드 검색, SQL 쿼리로 가져온 데이터를 LLM 컨텍스트에 전달



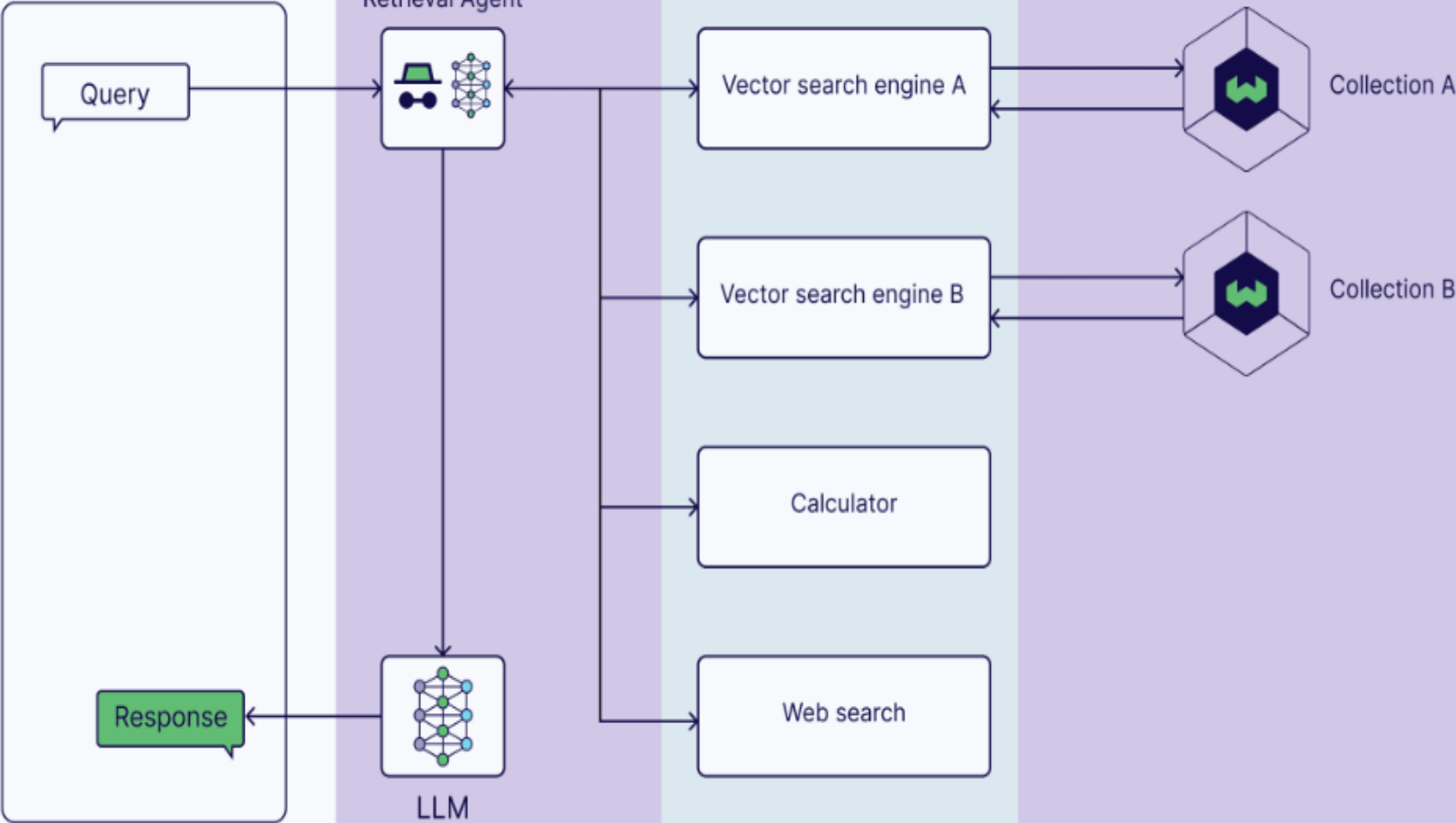
- 전통적 RAG

- “Retrieval → LLM”의 고정된 파이프라인
- 사용자의 질문이 단순하든 복잡하든 검색해서 컨텍스트를 LLM에 전달

- Agentic RAG : Agent + RAG

- 검색을 할지 말지 스스로 결정.
- 에이전트는 상황에 따라 어떤 데이터 소스를 쿼리할 지 결정.
- 복잡한 질문을 여러 단계로 나눠서 분석 및 검색하는 등 검색 전략을 실시간으로 조정할 수 있음.

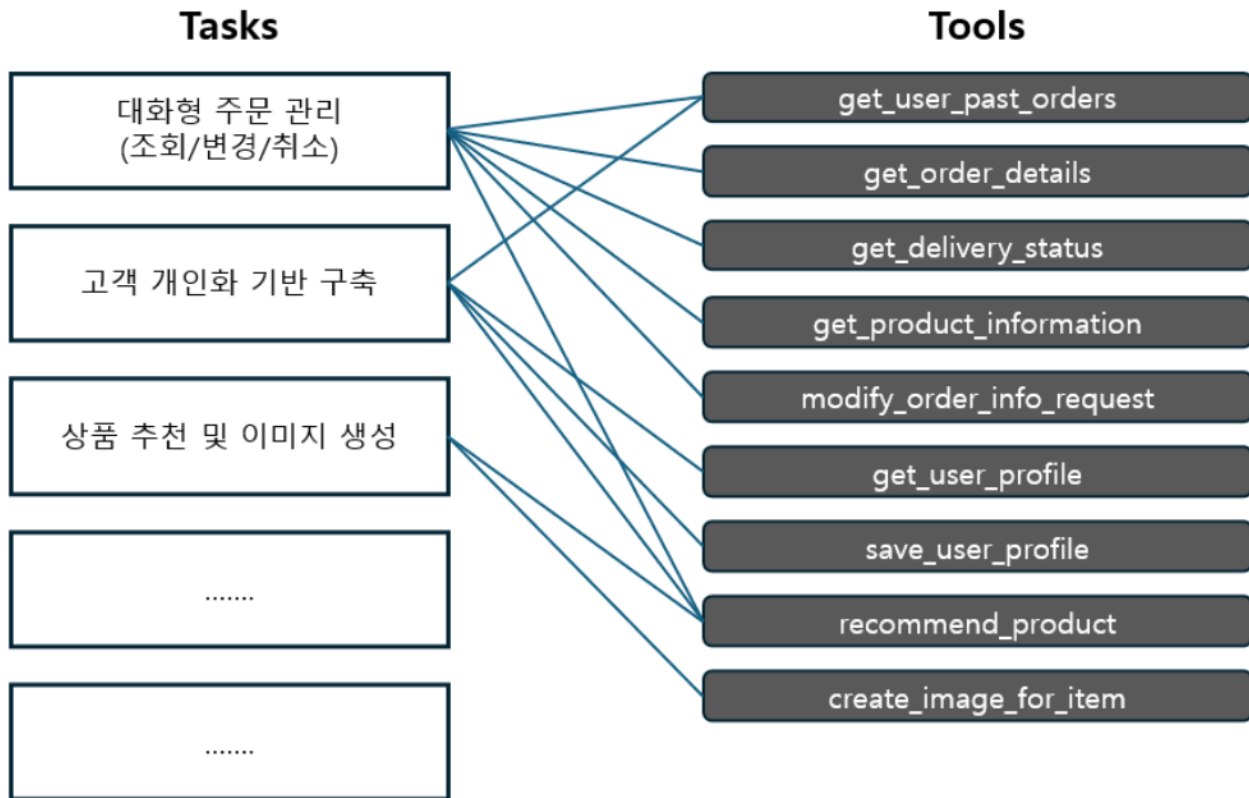
Single Agent RAG System (Router)



(2) System prompt

- 시스템 프롬프트는 AI의 직무 설명서
 - 자신이 누구여야 하고 어떻게 행동해야 하는지를 알려줌
 - AI의 성격, 지식 기반, 그리고 따라야 할 규칙들을 설정
- 구성요소
 - 페르소나 및 행동의 틀 부여
 - ✓ 에이전트가 누구인지, 어떤 톤과 매너로 소통해야 하는지 정의
 - 행동 가이드라인 및 제약 사항 설정
 - ✓ 에이전트가 행동할 수 있는 범위와 금지 사항을 명시
 - 도구 및 기능 활용 지침
 - ✓ 에이전트가 외부 도구(API, 브라우징, 코드 해석기 등)를 언제, 어떻게 사용해야 하는지 제공

작업 완수를 위해 여러 Tool 사용 필요



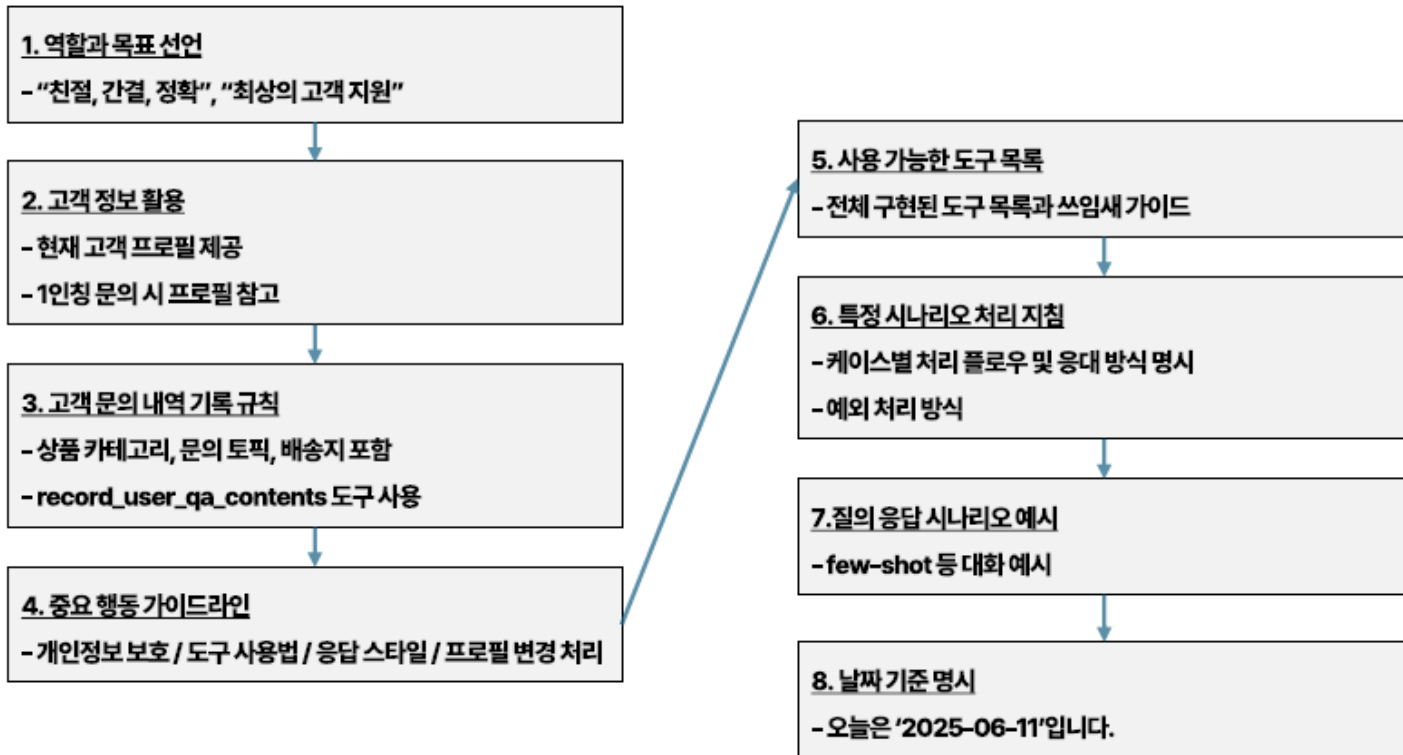
구분	명칭	기능 요약
주문 조회	get_user_past_orders()	고객의 전체 주문 내역(상품명, 수량, 상태 등) 조회
배송 상태 조회	get_delivery_status()	배송 상태, 이력, 예상 도착일 등 조회
주문+배송 조회	get_order_with_delivery()	주문 + 배송 정보 동시 조회
주문 생성	create_order()	상품 주문 생성 (사용자 ID, 상품 ID, 수량, 주소 입력 필요)
주문 변경	update_order()	수량/주소/상품 ID 수정 (결제 완료 상태만 가능)
주문 취소	cancel_order()	주문 취소 처리 (결제 완료 상태만 가능)
환불 요청	request_refund()	배송 완료 상태인 주문의 환불 요청 처리
교환 요청	request_exchange()	배송 완료 상태인 주문의 교환 요청 처리
상품 검색	search_products()	키워드 또는 가격 조건 기반 상품 검색
환불 규정 조회	get_refund_policy()	상품의 환불/교환 규정 텍스트 반환
단기 기억 관리	summarize_old_messages()	오래된 메시지를 요약 메시지 수/토론 수 최적화
요약 병합	merge_old_summaries()	복수 개 요약 메시지를 하나로 병합 컨텍스트 유지
프로필 로드	load_existing_profile()	기존 사용자 프로필 불러오기
프로필 추출	extract_user_profile()	대화에서 개인/쇼핑 정보 추출
프로필 병합	update_user_profile_via_llm()	기존 프로필 + 신규 추출 정보 통합
프로필 저장	save_profile_to_db()	병합된 사용자 프로필을 DB에 저장
.....		
.....		
.....		

도구는 하나의 명확한 의도(intent)를 수행하는 수준으로 설계
search_xxx(), analyze_xxx(), generate_xxx(), execute_xxx()

도구 제어를 위한 시스템 프롬프트 작성 예시

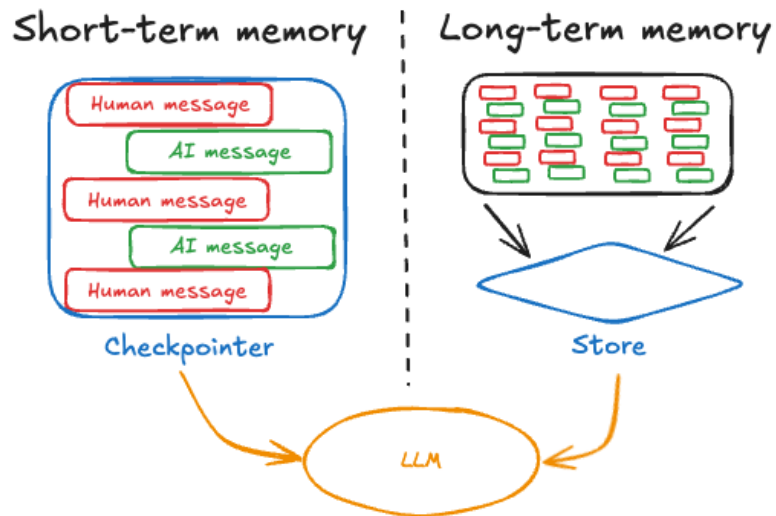


때때로 작업(Task)에 대한 Tool 실행의 순서를 AI에게 알려줘야 함.
e.g) 상품 취소는 어떻게 처리하면 좋을까?

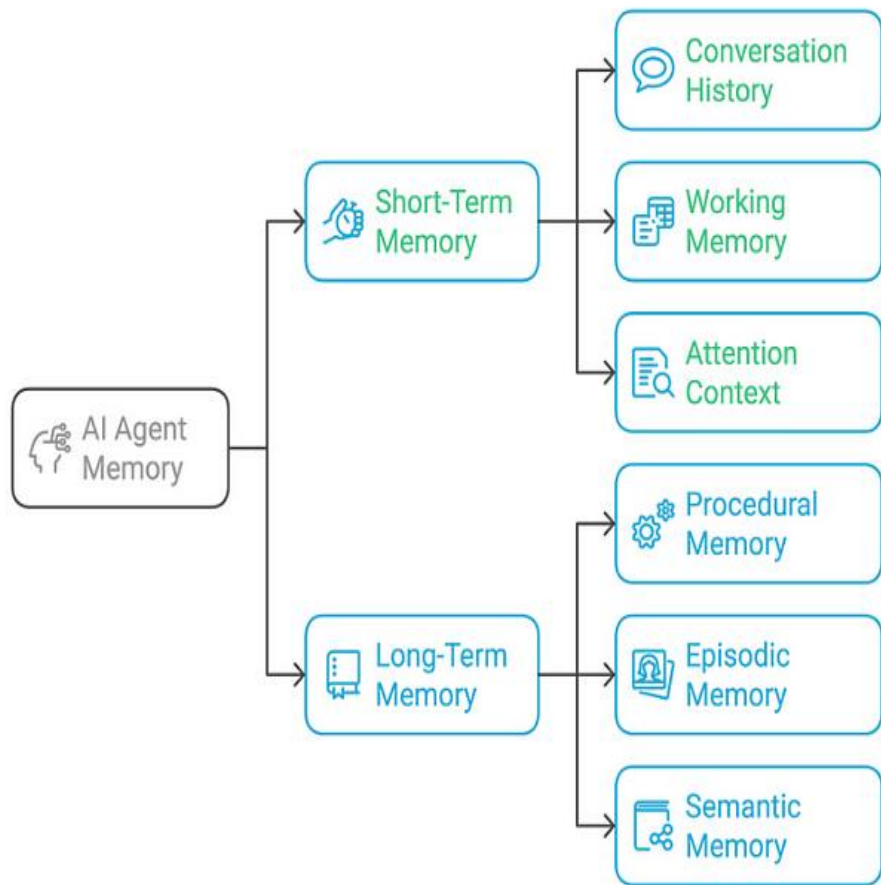


예시) 스마트 쇼핑 에이전트의 시스템 프롬프트 구성

(3) Memory



- 에이전트의 메모리는 시간축으로 확장된 기억 컨텍스트
 - 단기 기억: AI가 현재 진행하는 대화 동안 “유지” 기억
 - 장기 기억: AI가 여러 대화에서 축적한 “학습” 기억



대화 기록: 최근 메시지 및 순서

작업 메모리: 임시 변수 및 상태

주의 맥락: 대화의 현재 초점

절차적 메모리

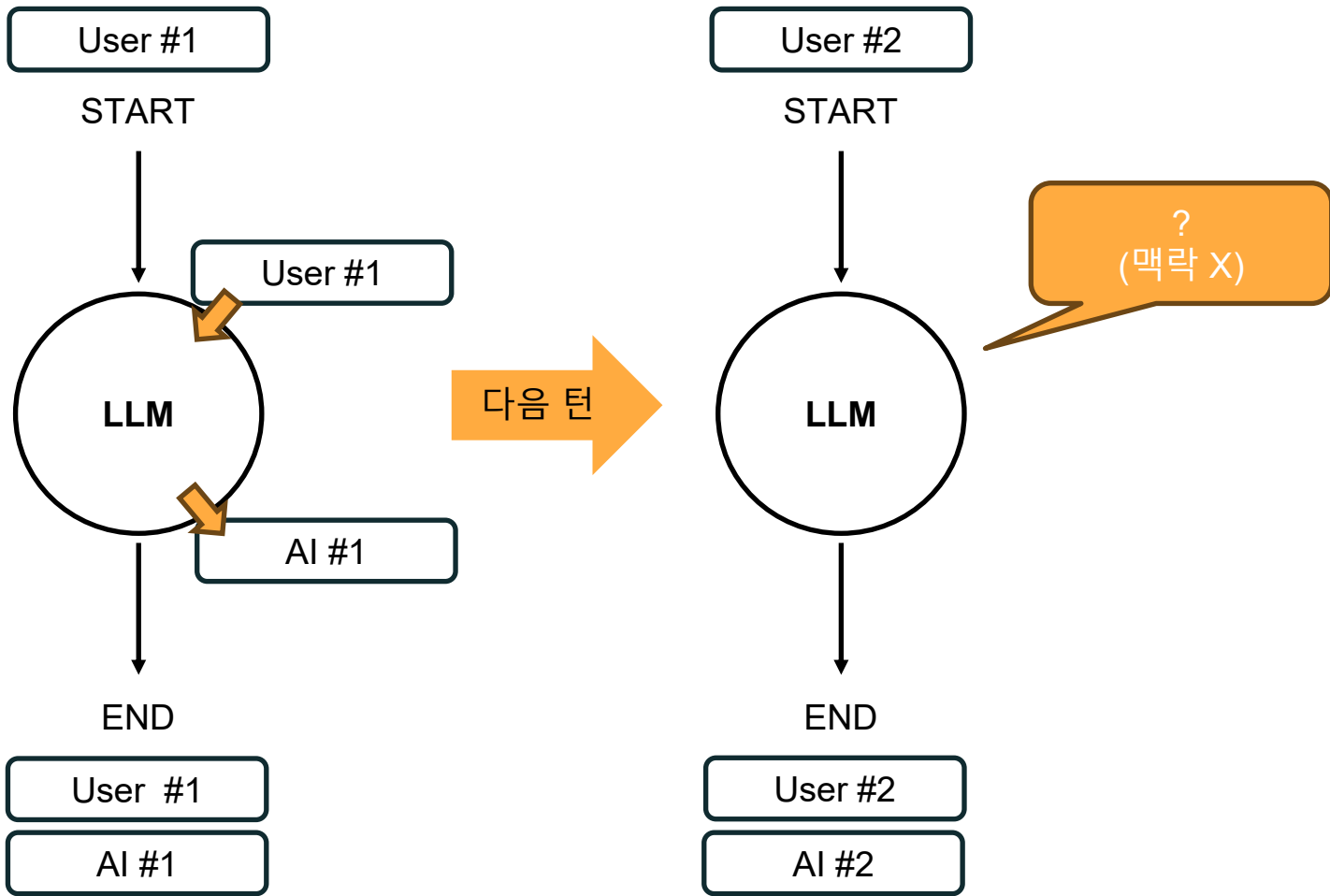
"API 문서 관련 이메일은 항상 우선순위에 두세요" 같은 학습된 행동

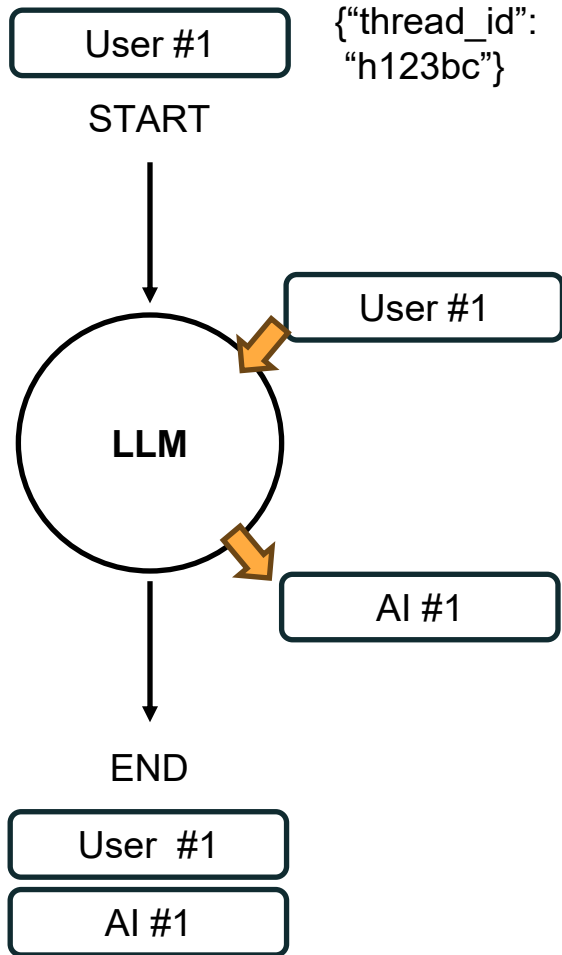
에피소드 메모리

"지난번에 이 고객에게 이메일을 보냈을 때, 제 답변이 너무 딱딱해서 마찰을 일으켰습니다" 같이 과거 사건과 상호작용에 대한 구체적인 기억

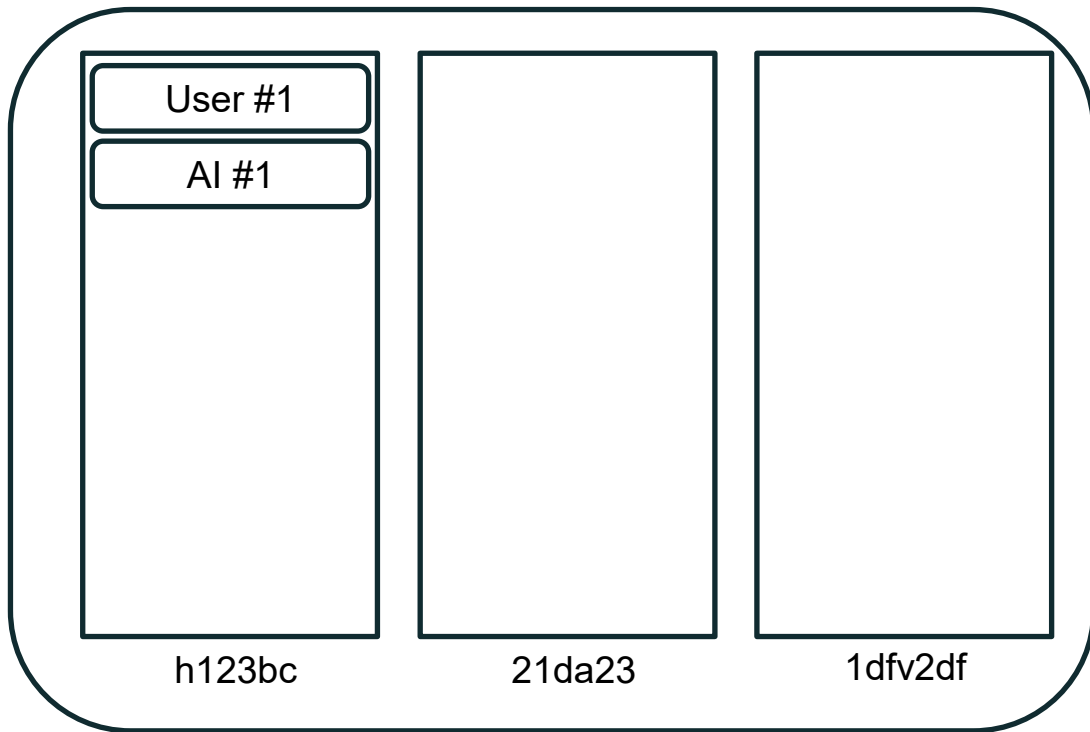
의미 기억

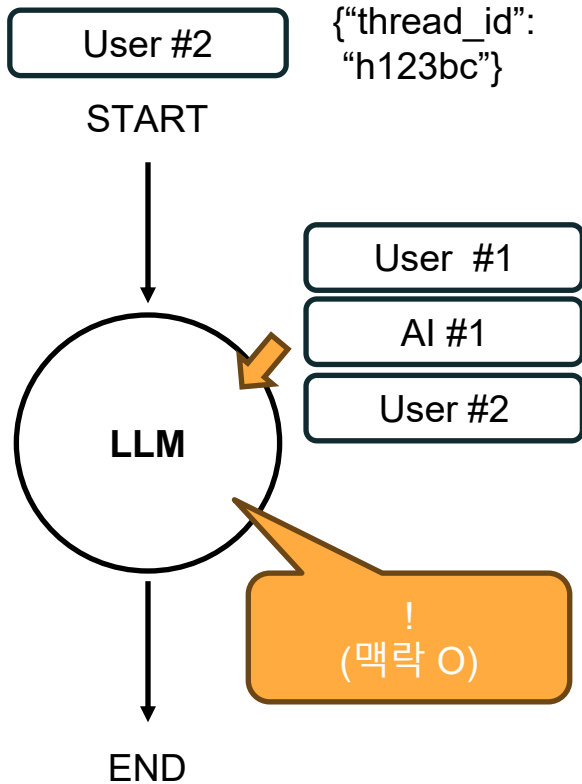
"앨리스는 API 문서 담당자입니다" 와 같이 사실을 기억



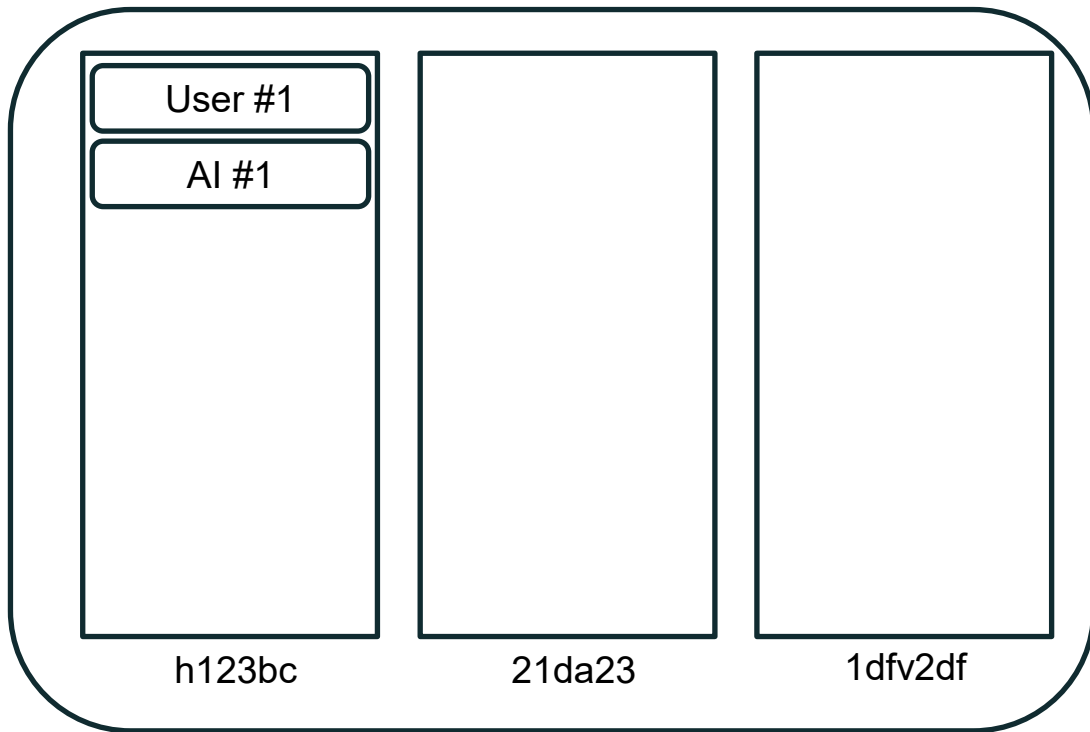


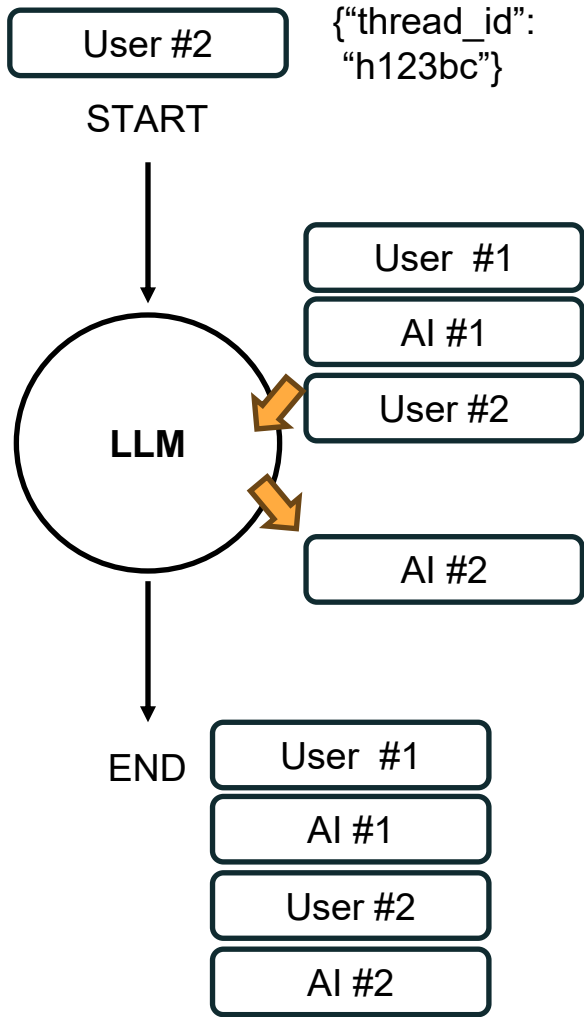
Checkpoint : LangGraph의 단기메모리



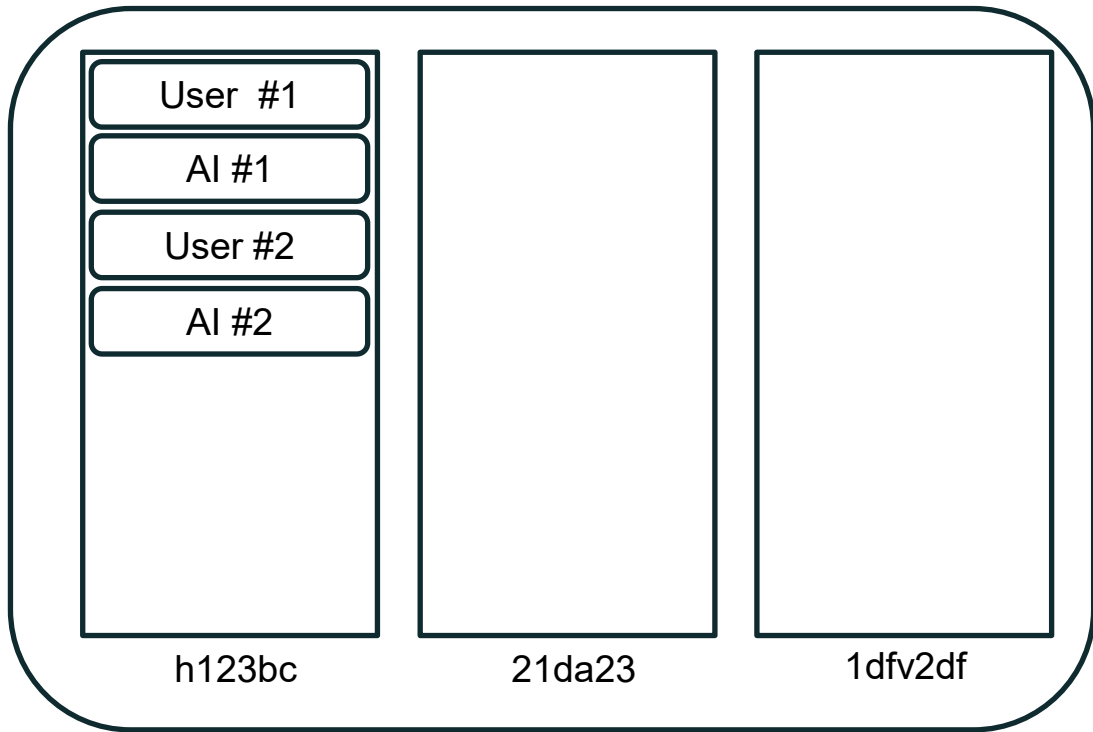


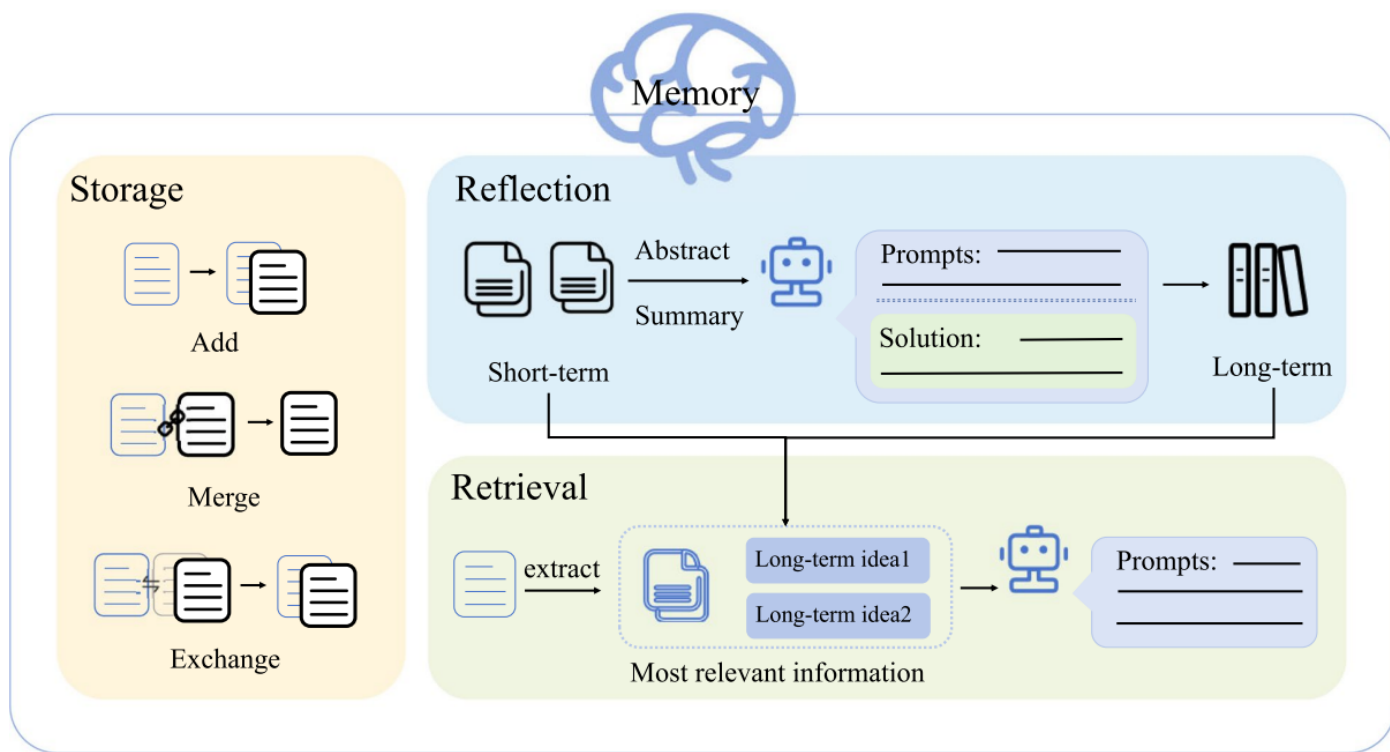
Checkpoint : LangGraph의 단기메모리





Checkpoint : LangGraph의 단기메모리





- **Reflection (추론):** Short-term memory의 파편화된 정보들을 요약하거나 추상화하여 핵심 정보를 추출
- **Storage (저장):** 추출된 지식을 장기 메모리에 저장할 때 병합, 추가, 교체하며 최적의 상태를 유지
- **Retrieval (인출):** 새로운 프롬프트가 입력되면 장기 기억에서 가장 관련된 정보를 검색해서 컨텍스트에 주입

추출된 기억은
장기메모리에 저장

추출된 사실

사용자는 연남동에 거주함

저장된 선호도

사용자는 존댓말
답변을 선호함

추출된 사실

사용자의 최애 커피는
아이스 아메리카노임

요약된 기억

문맥이 요약됨

대화 맥락이 달라져도
저장된 사실을 기억 가능

thread_id가 다른 별개의
대화에서도 기억 가능

사용된 사실

위치: 연남동

최애 커피: 아이스
아메리카노



오늘 어떤 커피를 도와드릴까요?

안녕하세요, 저는 연남동에 살고 커피를 아주 좋아해요.



안녕하세요, 김연아님! 연남동에 사시는 분을 만나서 반가워요.

답변하실 때 항상 존댓말을 써주시겠어요?



알겠습니다 - 이제부터는 정중하게 존댓말을 사용할게요!

네, 감사합니다... 좋아요!



별말씀을요, 마음에 드셔서 다행입니다.

그런데 제 최애 커피는 아이스 아메리카노예요!



꼭 기억하겠습니다!

...계속되는 긴 대화...



...계속되는 긴 대화...

지금 근처에 추천할 만한 곳이
있나요?



좋은 소식이에요 - 연남동의 아름다운 하늘 아래,
연아님이 아주 좋아하시는 시원한 아이스
아메리카노가 맛있는 카페가 바로 근처에 있어요!
주말 특별 블렌딩 정보도 드릴까요?

메시지 보내기...



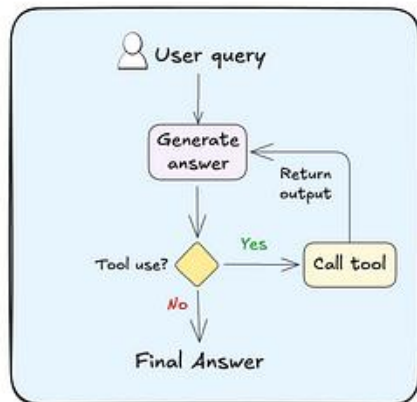
에이전트 설계 요소

설계 요소	목적	주요 내용
1. Agent Overview (에이전트 개요서)	전체 시스템의 목적 및 역할 정의	에이전트 이름, 목표, 주요 기능, 사용 시나리오, 상호작용 흐름
2. Architecture Specification (아키텍처 명세서)	시스템 구조와 구성요소 명확화	LLM, Tool, Memory, Context, Workflow, API 간 관계도
3. Prompt Design (프롬프트 설계서)	모델이 따를 지시 구조를 명확히 정의	동작 시나리오 기반 System/Tool Prompt, Few-shot 예시, 가이드라인(금지사항), 응답 스타일.
4. Context & Memory Design	상태 유지 및 정보 관리 전략 수립	Memory 유형(Key-value, Embedding 등), 보존·갱신 규칙
5. Tool Specification (도구 명세서)	외부 시스템·API 연동 정의	각 Tool의 이름, Input/Output Schema, 호출 조건, 예시 응답
6. Workflow Diagram (흐름도)	에이전트의 의사결정 및 호출 흐름 시각화	Plan → Action → Observation → Reflection 루프 구조
7. Evaluation Plan (평가 기준서)	에이전트 성능 검증 체계 수립	정량/정성 지표, 성공 조건, 테스트 케이스
8. Logging & Feedback Policy (로그/피드백 정책)	동작 추적 및 개선을 위한 기준 설정	대화 로그 구조, 피드백 수집 방식, 개선 주기
9. Safety & Control Policy (안전성 정책서)	예외 처리와 윤리적 통제 설정	금지 응답 규칙, 실패 시 fallback 동작, 예외 처리 로직

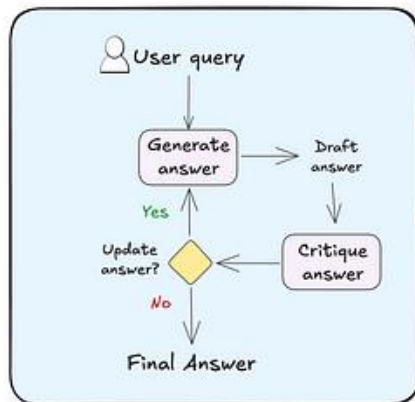
부록. Agentic patterns

주요 Agentic Patterns

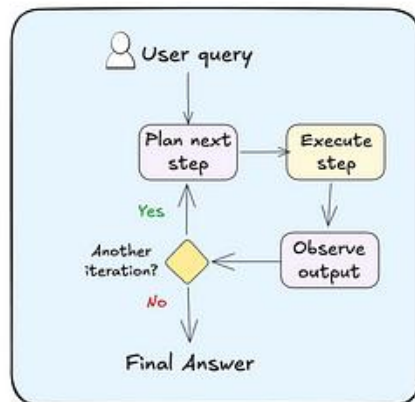
Tool Use



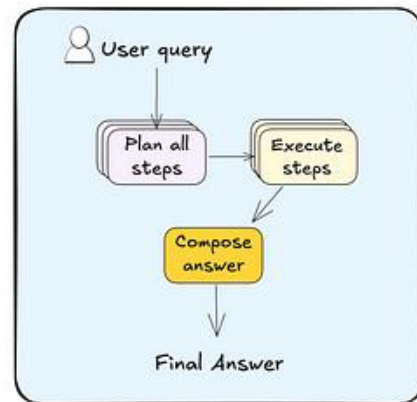
Basic Reflection



ReAct

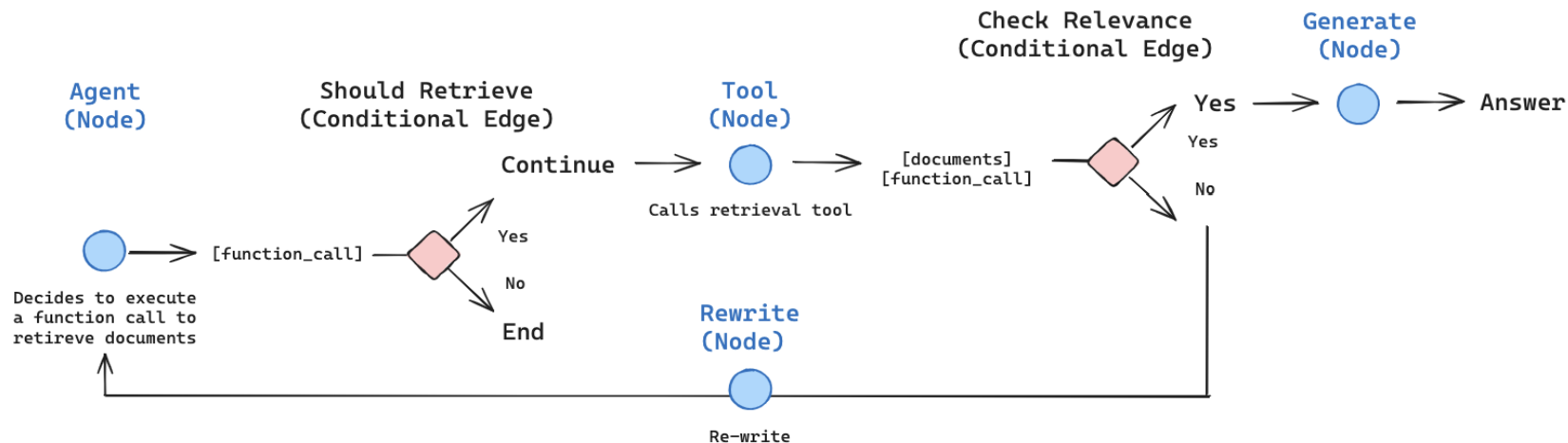


Plan-then-Execute



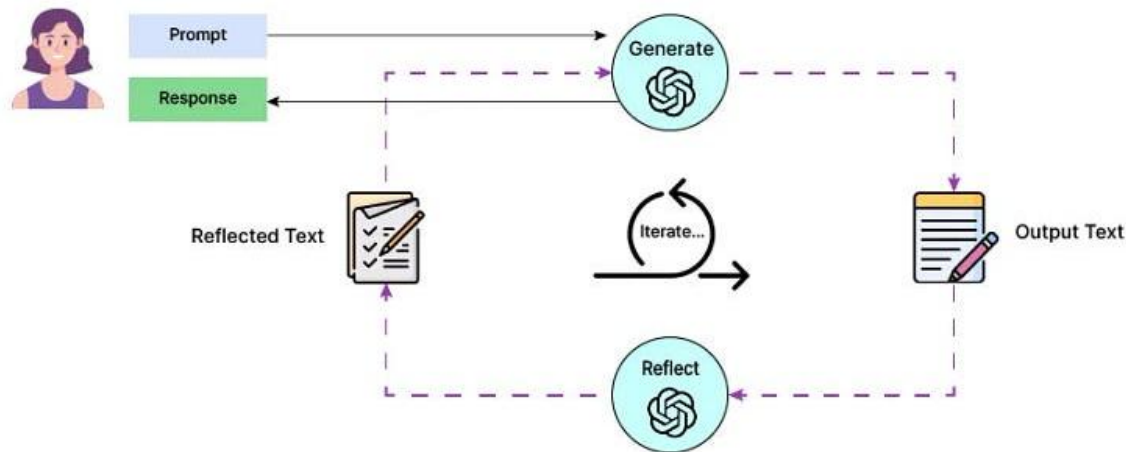
Legend LLM Tool Orchestration logic

Agentic RAG



- 단순한 검색 후 응답 생성이 아닌 Agent가 의사 결정을 내리고 상태(State)를 유지하면서, 검색(Retrieval), 재작성(Query Rewrite), 응답 생성(Generation) 등의 다양한 작업을 수행할 수 있는 프레임워크

Reflection pattern



```
from langgraph.graph import MessageGraph

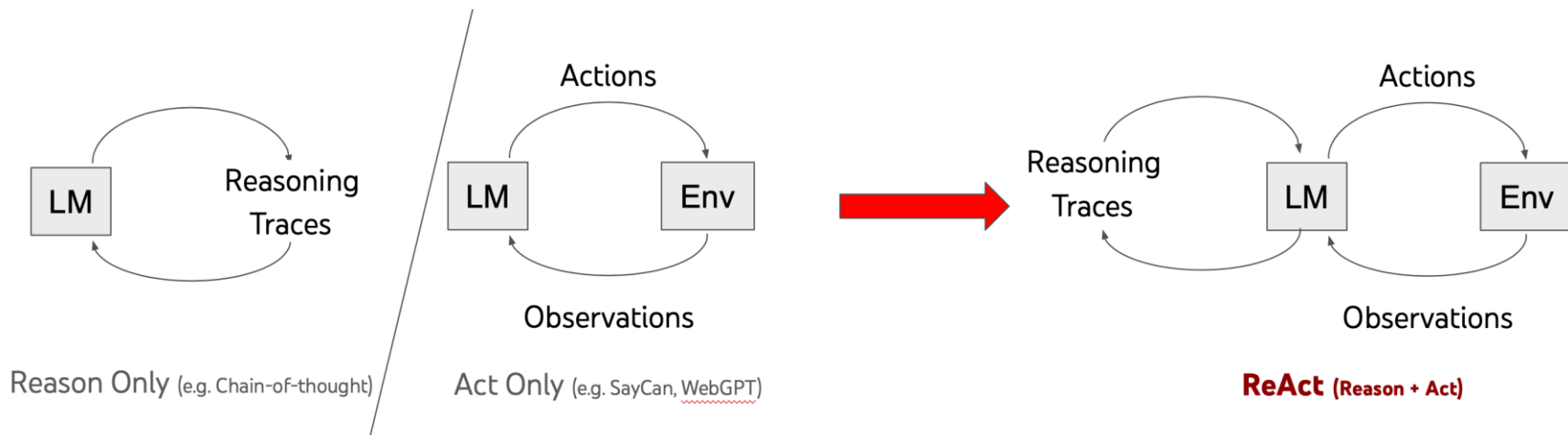
builder = MessageGraph()
builder.add_node("generate", generation_node)
builder.add_node("reflect", reflection_node)
builder.set_entry_point("generate")

def should_continue(state: List[BaseMessage]):
    if len(state) > 6:
        return END
    return "reflect"

builder.add_conditional_edges("generate", should_continue)
builder.add_edge("reflect", "generate")
graph = builder.compile()
```

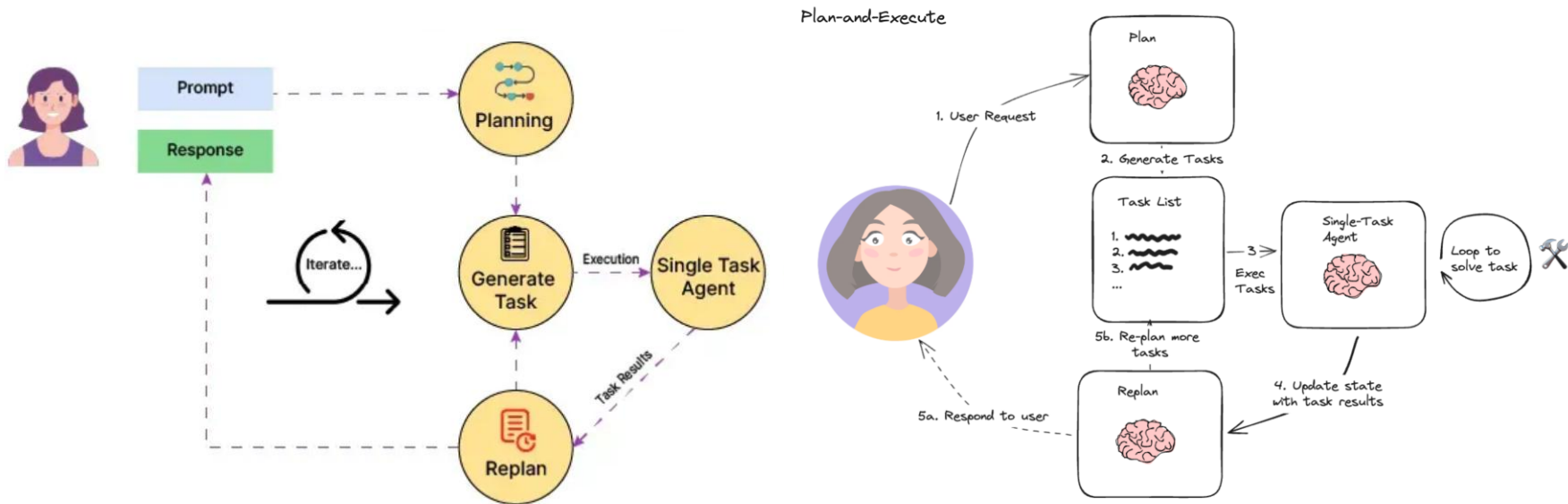
- AI Agent가 자신의 행동을 평가하고 피드백을 반영하여 스스로 개선하는 방식
- 과거의 성공과 실패를 분석해 학습, 반복 수행을 통해 점점 더 정교한 답변 생성
- 계산 비용 증가 / 느린 피드백 루프는 단점이 될 수 있음.

Reason-then-Act (ReAct)



- 에이전트가 상황에 대한 추론(Think) → 실행할 행동을 결정하고 실행(Act)
- 행동하기 전에 충분히 검토해 논리적인 결론을 내린 후 실행

Plan-then-Execute (planning agent)



- AI Agent가 먼저 전체적인 계획(Plan)을 수립한 후, 실행(Act)하는 방식
- 계획 단계에서 목표를 설정, 필요한 작업을 정의 후 실행 단계에서 이를 수행
- 유연성 부족 / 초기 계획 오류 / 계산 비용 증가는 단점이 될 수 있음.

Thank you!